05 - Görev Zamanlayıcı Değerlendirme

Amaç ve Kapsam

Bu notebook, tezin donanımsal zamanlama algoritmalarının başarısını özetleyen en kritik analiz bölümünü temsil etmektedir. Önceki veri hazırlama ve modelleme adımlarında (Notebook 01-04) özenle geliştirilen ve eğitilen **tüm ağaç tabanlı makine öğrenmesi modelleri (Random Forest, XGBoost, LightGBM)** ve **derin öğrenme modelleri (CNN, LSTM, CNN-LSTM)**, bu aşamada gerçeğe yakın bir küme (cluster) simülasyon ortamında test edilmektedir. Profesyonel MLOps süreçlerine (Model Ops) uygun olarak; modeller her defasında baştan eğitilmek yerine, önceden diske kaydedilmiş (serialized) `.joblib` ve `.pth` objelerinden doğrudan okunarak canlı tahmine (inference) hazır hale getirilmektedir.

Bu analizde, zamanlama politikalarının başarımını tam anlamıyla kıyaslayabilmek için aşağıdaki senaryolar (kapsayıcı ana kategoriler) kurgulanmıştır:

Kategori 1 — Temel Çizgiler ve Üst Sınırlar (Baselines & Upper Bounds):

- **1. FIFO (First-In-First-Out):** İşlerin herhangi bir akıllı önceliklendirme yapılmadan katı bir şekilde geliş sıralarına göre kuyruğa alındığı, endüstride yaygın olarak kullanılan taban (baseline) yöntem.
- **2. SJF-Oracle (Shortest-Job-First - Kusursuz Bilgi):** İş tamamlama sürelerinin önceden %100 doğrulukla bilindiği varsayılan, performansın teorik üst sınırını (ideal senaryoyu) temsil eden yapı.

Kategori 2 — Ağaç Tabanlı Modeller (Deney A & B):

- **3. SJF-Numeric (Temel ML):** Yalnızca sayısal donanım metrikleriyle eğitilmiş, zamandan bağımsız makine öğrenmesi modellerinin (Random Forest, LightGBM, XGBoost) çalışma süresi tahminlerine dayalı çizelgeleme simülasyonları.
- **4. SJF-Categorical (Gelişmiş ML):** Ağaç algoritmaları için, sayısal özelliklerin yanı sıra benzersiz kodlama yöntemlerini kullanarak yüksek kardinaliteye sahip kategorik değişkenleri (örn. `user`, `gpu_type`) de kapsayan gelişmiş simülasyon senaryoları.

Kategori 3 — Statik Derin Öğrenme (Deney C & D):

- **5. Static DL (Sayısal & Kategorik):** PyTorch tabanlı Derin Öğrenme mimarilerinin (1D-CNN, LSTM ve Hibrit CNN-LSTM), herhangi bir zamansal bağlantı kurmadan geleneksel tablo (satır) verisine anlık ileri besleme yaparak tahmin ürettiği senaryo.

Kategori 4 — Zaman Serisi Kayan Pencere DL Sürümleri (Deney E & F):

- **6. Sequence DL (Sayısal & Kategorik):** Açık bir geçmiş kayan penceresi (sliding window, `seq_len=10`) kullanan nihai "State-of-the-Art" (En güncel teknoloji) uygulama. Burada PyTorch mimarileri; hem yerel işlemsel karmaşıklığı (CNN filtreleriyle) hem de uzun vadeli kuyruk/bekleme kısıtlarını (tekrarlayan LSTM katmanlarıyla) yakalamak için ardışık iş durumlarının dizilerini eşzamanlı işler. Hem sayısal hem kategorik değişkenler için değerlendirilir.

Temel Hedef: Zamandan bağımsız anlık makine öğrenmesi algoritmalarının, zamansal "kayan pencere (sliding-window)" kullanan Derin Öğrenme motorlarıyla (CNN, LSTM, Hibrit) yan yana getirildiğinde sistem bekleme sürelerini (Wait Time) ve İş Tamamlanma Sürelerini (JCT / Turnaround Time) geleneksel kuyruk politikalarına kıyasla ne kadar azalttığını istatistiksel ve bilimsel olarak kanıtlamaktır.

Literatür ve Gerçek Dünya Mimarileri (Google Borg, Kubernetes, Apache YARN)

Büyük veri merkezlerinde ve Yüksek Başarımlı Hesaplama (HPC) kümelerinde **Görev Zamanlama (Job Scheduling)** problemi, milyar dolarlık altyapıların karlılığını belirleyen en kritik algoritmik savaştır. Gerçek dünyada bu sistemler nasıl yönetilir?

- **Google Borg & Kubernetes (K8s):** Google'ın iç sistemlerinde on yılı aşkın süredir kullandığı *Borg* ve onun açık kaynaklı halefi olan *Kubernetes*, konteyner (container) tabanlı iş yüklerini dağıtır. Bu sistemler, gelen işlerin CPU/RAM/GPU taleplerini (requests/limits) önceden bilir, ancak işin **ne zaman biteceğini (runtime) bilmezler**. Geleneksel olarak, bu sistemler QoS (Quality of Service) sınıflarına göre statik önceliklendirme yapar.
- **Apache YARN (Hadoop):** Büyük veri ekosistemlerinde kullanılan YARN, *Kapasite Zamanlayıcısı (Capacity Scheduler)* veya *Adil Zamanlayıcı (Fair Scheduler)* kullanır. Tahminleme yeteneği sınırlıdır; uzun süren bir Spark görevini önceden tespit edemez.
- **Slurm & PBS:** Genellikle süper bilgisayarlarda (HPC) ve akademik kümelerde kullanılan Slurm, **En Kısa İş İlk (Shortest Job First - SJF)** mantığını **Geri Doldurma (Backfilling)** yaklaşımıyla harmanlar. Ancak, kullanıcıların girdikleri tahmini süreler genellikle çok yanlıştır (kullanıcılar işleri kesilmesin diye hep en uzun süreyi yazar).

> 💡 **Bu Tezin Endüstriyel Değeri:** Zamanlama süreçlerinde güvenilmez ve eksik olan **kullanıcı beyanlarına (user estimates)** bağımlı kalmak yerine; Makine Öğrenmesi ve Derin Öğrenme mimarileri sisteme bir **Oracle (Kâhin)** olarak entegre edilmiştir. Modellerin milisaniyeler seviyesinde ürettiği bu öngörüler (Predicted Runtime), GPU kümelerinin körü körüne değil; verilere dayalı, dinamik, agresif ve sıfır-darboğaz hedefiyle akıllıca yönetilmesini sağlar.

Kuyruk Teorisi (Queueing Theory) ve Yük Faktörü (Load Factor)

Simülasyonumuzun kalbinde, **Kuyruk Teorisi** yatmaktadır. Spesifik olarak, GPU kümemiz M/M/c veya literatürdeki daha karmaşık dağılımlara uyacak heterojen çoklu-sunucu (multi-server) modelidir.

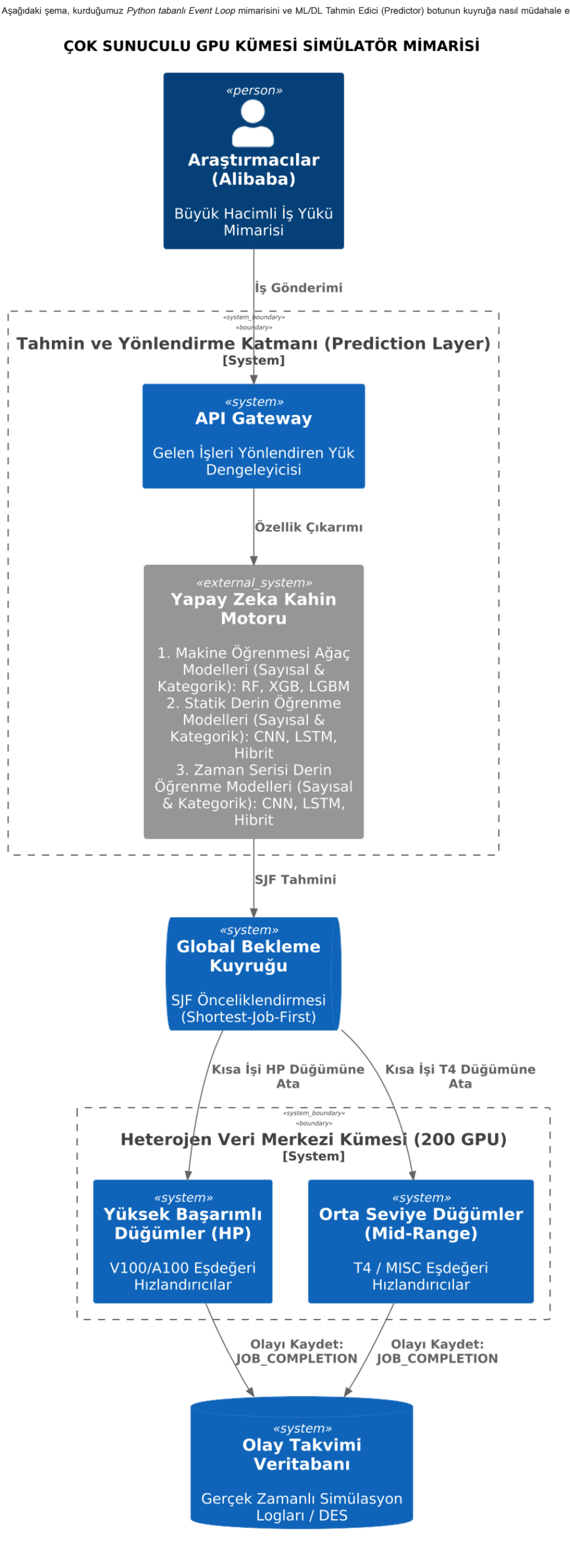
- **Varış Oranı (Arrival Rate - λ):** Kümeye birim zamanda gelen işlerin hızı. Alibaba verisetinde bu çok değişkendir (bursty traffic).
- **Servis Oranı (Service Rate - μ):** Kümelerdeki GPU'ların işleri bitirme hızı.
- **Yük Faktörü (Load Factor - ρ):** Sistemin üzerindeki genel stres. Eşik değeri olan $\rho \approx 1.0$ aşıldığında işler kuyrukta sonsuza kadar beklemeye başlar.

Eğer modellerimiz (XGBoost, LSTM, vb.) kısa sürecek işleri isabetli şekilde kuyruğun en önüne alabilirse (SJF mantığı), **Bekleme Süresi (Wait Time)** makroskopik olarak ciddi oranda düşecektir.

Ayrık Olay Simülasyonu (Discrete-Event Simulation) Mimarisi

Bu notebook'ta çalıştıracağımız kod, zamanı saniye saniye sayan naif bir döngü değil; endüstri standardı olan **Ayrıntılı Olay Simülatörü (DES)**'dur. Simülatör, zaman ekseninde yalnızca `JOB_ARRIVAL` (İş Gelişi) ve `JOB_COMPLETION` (İş Bitişi) anlarına sıçrayarak çalışır.

Aşağıdaki şema, kurduğumuz *Python tabanlı Event Loop* mimarisini ve ML/DL Tahmin Edici (Predictor) botunun kuyruğa nasıl müdahale ettiğini özetlemektedir:


ÇOK SUNUCULU GPU KÜMESİ SİMÜLATÖR MİMARİSİ
«person»
Araştırmacılar (Alibaba)
Büyük Hacimli İş Yükü Mimarisi
İş Gönderimi
«system_boundary»
«boundary»
Tahmin ve Yönlendirme Katmanı (Prediction Layer)
[System]
«system»
API Gateway
Gelen İşleri Yönlendiren Yük Dengeleyicisi
Özellik Çıkarımı
«external_system»
Yapay Zeka Kahin Motoru
1. Makine Öğrenmesi Ağaç Modelleri (Sayısal & Kategorik): RF, XGB, LGBM
2. Statik Derin Öğrenme Modelleri (Sayısal & Kategorik): CNN, LSTM, Hibrit
3. Zaman Serisi Derin Öğrenme Modelleri (Sayısal & Kategorik): CNN, LSTM, Hibrit
SJF Tahmini
«system»
Global Bekleme Kuyruğu
SJF Önceliklendirmesi (Shortest-Job-First)
Kısa İşi HP Düğümüne Ata
Kısa İşi T4 Düğümüne Ata
«system_boundary»
«boundary»
Heterojen Veri Merkezi Kümesi (200 GPU)
[System]
«system»
Yüksek Başarımlı Düğümler (HP)
V100/A100 Eşdeğeri Hızlandırıcılar
«system»
Orta Seviye Düğümler (Mid-Range)
T4 / MISC Eşdeğeri Hızlandırıcılar
Olayı Kaydet: JOB_COMPLETION
Olayı Kaydet: JOB_COMPLETION
«system»
Olay Takvimi Veritabanı
Gerçek Zamanlı Simülasyon Logları / DES

Zaman Serisi (Sequence) Modellerinin Simülasyona Entegrasyonu (Kayan Pencere)

Geleneksel tablo biçimli makine öğrenmesi algoritmaları (Exp A, B) ve tek adımlı derin öğrenme algoritmaları (Exp C, D) anlık veriye bakıp tahmin yürütür.

Ancak, **Deney E ve F (Zaman Serisi Derin Öğrenme Modelleri)**, tahmin yapabilmek için `seq_len = 10` boyutunda bir zaman hafızasına ihtiyaç duyar. Gerçek dünyadaki bir simülatör zaman içinde lineer aktığı için gelecek veriye bakılamaz.

> **Dinamik Bellek Mimarisi / Dynamic Sliding Window:**
>
> Simülatör kodunun içerisine bir 'Zaman Serisi Tampon Bellek' tasarlanmıştır. `predict_sequence_baseline` mantığı, o anki simülasyon zamanından geriye doğru gelen en son 9 işi (kuyruktan veya bitmiş işlerden) alır, güncel işi de 10. adım olarak sonuna ekler. Böylece 3-boyutlu `(1, 10, features)` veri yapısı milisaniyeler içinde inşa edilir ve **LSTM / CNN / CNN-LSTM** modellerinin içine enjekte edilir.

In [1]:

```python
# ── 0. Environment & Path Setup ──────────────────────────────────────────────
import os
import sys
import warnings
from pathlib import Path

# ── KRİTİK: torch/xgboost/sklearn import edilmeden ÖNCE ayarlanmalı ──────────
# PyTorch ve XGBoost macOS'ta kendi libiomp/libomp kütüphanelerini yükler.
# KMP_DUPLICATE_LIB_OK ayarlanmazsa işlem hard segfault ile çöker (kernel crash).
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "1"          # tek iş parçacıklı OpenMP çakışmayı önler

# MPS / CUDA güvenliği
os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"
os.environ["PYTORCH_MPS_HIGH_WATERMARK_RATIO"] = "0.0"
os.environ["PYTORCH_NO_CUDA_MEMORY_CACHING"] = "1"
os.environ["TOKENIZERS_PARALLELISM"] = "false"

warnings.filterwarnings("ignore")

PROJECT_ROOT = Path().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

print(f"[Setup] Project root : {PROJECT_ROOT}")
print(f"[Setup] Python path  : {sys.executable}")
```

```
[Setup] Project root : /Users/hasanugurcelebi/Thesis/alibaba-gpu-runtime-prediction-and-scheduling
[Setup] Python path  : /Users/hasanugurcelebi/Thesis/alibaba-gpu-runtime-prediction-and-scheduling/venv/bin/python
```

In [2]:

```python
# ── 1. Imports ───────────────────────────────────────────────────────────────
# stdlib / third-party
import pandas as pd
import numpy as np
import joblib
import torch
import torch.nn as nn
from IPython.display import display
from copy import deepcopy
import matplotlib.pyplot as plt
import seaborn as sns

DATA_DIR = PROJECT_ROOT / "data"
MODEL_DIR = PROJECT_ROOT / "results" / "models"

from src.feature_engineering import prepare_features_for_model

from src.simulation.multi_node_simulator import MultiNodeClusterSimulator, provision_heterogeneous_gpu_cluster
from src.simulation.scheduler_simulator import FIFOScheduler, SJFScheduler, SJFPredScheduler, SRFScheduler


# CPU — yukarıda ayarlanan OMP_NUM_THREADS=1 ile eşleşmesi için tek iş parçacığı
device = torch.device('cpu')
torch.set_num_threads(1)
print('PyTorch Device Setup:', device)
print('PyTorch version     :', torch.__version__)
```

```
PyTorch Device Setup: cpu
PyTorch version     : 2.11.0
```

Kaydedilmiş Modellerin Yüklenmesi

Önceki notebook'ta (Notebook 04) hiperparametre optimizasyonu (tuning) tamamlanan ve sonlandırılan en iyi modeller, diske `models/` dizinine `.joblib` formatında kaydedilmişti. Bu adımda, eğitim süresi maliyetini tamamen ortadan kaldırmak ve üretim (production) performansını simüle etmek amacıyla modeller doğrudan diskten belleğe yüklenmektedir.

In [3]:

```python
import gc
import joblib
# ── 2. Load tree-based models ────────────────────────────────────────────────
print("[Step 1] Loading serialized models from disk...")

# Sayısal ağaç modelleri
rf_num  = joblib.load(MODEL_DIR / "rf_numeric.joblib")
xgb_num = joblib.load(MODEL_DIR / "xgb_numeric.joblib")
lgb_num = joblib.load(MODEL_DIR / "lgbm_numeric.joblib")
gc.collect()

# Kategorik ağaç modelleri
rf_cat      = joblib.load(MODEL_DIR / "rf_categorical.joblib")
xgb_cat     = joblib.load(MODEL_DIR / "xgb_categorical.joblib")
lgb_cat_nat = joblib.load(MODEL_DIR / "lgbm_categorical_native.joblib")
gc.collect()

# DL ölçekleyiciler
lstm_scaler_x     = joblib.load(MODEL_DIR / "lstm_scaler_x.joblib")
lstm_scaler_y     = joblib.load(MODEL_DIR / "lstm_scaler_y.joblib")
lstm_scaler_x_cat = joblib.load(MODEL_DIR / "lstm_scaler_x_cat.joblib")
lstm_scaler_y_cat = joblib.load(MODEL_DIR / "lstm_scaler_y_cat.joblib")
gc.collect()

print("[Step 1] Ağaç modelleri ve ölçekleyiciler yüklendi.")

# ── 2.5 Derin Öğrenme Model Ağırlıklarını Yükle (.pth) ───────────────────────
print("[Step 1.5] DL modelleri diskten yükleniyor (grup grup)...")
from src.models.dl_runtime_predictor import RuntimePredictorCNN, RuntimePredictorLSTM, RuntimePredictorCNNLSTM

def _load(path):
    """Tam PyTorch model nesnesini CPU'ya yükle, grad izlemesini kapat."""
    m = torch.load(path, map_location="cpu", weights_only=False)
    m.eval()
    for p in m.parameters():
        p.requires_grad_(False)
    return m

try:
    # Grup 1 — Statik sayısal (Exp C)
    cnn_static_num    = _load(MODEL_DIR / "cnn_numeric.pth")
    lstm_static_num   = _load(MODEL_DIR / "lstm_numeric.pth")
    hybrid_static_num = _load(MODEL_DIR / "cnn_lstm_numeric.pth")
    gc.collect()

    # Grup 2 — Statik kategorik (Exp D)
    cnn_static_cat    = _load(MODEL_DIR / "cnn_categorical_pt.pth")
    lstm_static_cat   = _load(MODEL_DIR / "lstm_categorical_pt.pth")
    hybrid_static_cat = _load(MODEL_DIR / "cnn_lstm_categorical_pt.pth")
    gc.collect()

    # Grup 3 — Sayısal dizi (Exp E)
    cnn_model_num_seq    = _load(MODEL_DIR / "cnn_numeric_seq.pth")
    lstm_model_num_seq   = _load(MODEL_DIR / "lstm_numeric_seq.pth")
    hybrid_model_num_seq = _load(MODEL_DIR / "cnn_lstm_numeric_seq.pth")
    gc.collect()

    # Grup 4 — Kategorik dizi (Exp F)
    cnn_model_cat_seq    = _load(MODEL_DIR / "cnn_categorical_seq.pth")
    lstm_model_cat_seq   = _load(MODEL_DIR / "lstm_categorical_seq.pth")
    hybrid_model_cat_seq = _load(MODEL_DIR / "cnn_lstm_categorical_seq.pth")
    gc.collect()

    print("[SUCCESS] 12 PyTorch modeli CPU'ya yüklendi ve donduruldu!")
except Exception as e:
    print(f"DL Yükleme Hatası: {e}")
```

```
[Step 1] Loading serialized models from disk...
[Step 1] Ağaç modelleri ve ölçekleyiciler yüklendi.
[Step 1.5] DL modelleri diskten yükleniyor (grup grup)...
[SUCCESS] 12 PyTorch modeli CPU'ya yüklendi ve donduruldu!
```

Simülasyon için Verilerin Yüklemesi

Bütünlüğün ve makine öğrenmesi ve derin öğrenmesi etiğinin korunması (data leakage'ı önlemek) amacıyla, analiz daha önce algoritmaların hiç görmediği o orijinal `%20`'lik test veri kümesi üzerinde gerçekleştirilmelidir. Bunun sağlanması için `prepare_features_for_model` fonksiyonu kullanılarak ana yapıdan test kümeleri (`X_test_num`, `X_test_cat_oh`, `X_test_cat_base`) eksiksiz olarak yeniden oluşturulur ve indekslerle bağlanır.

In [4]:

```python
# ── 3. Load numeric-only test set ────────────────────────────────────────────
print("[Step 2] Loading numeric-only test set...")
(job_df, _, X_test_num, _, y_test_num, _, _) = prepare_features_for_model(
    dataset="main", time_unit="s", test_size=0.20, random_state=42,
    feature_mode="numeric_only",
)
print(f"  X_test_num shape: {X_test_num.shape}")
```

```
[Step 2] Loading numeric-only test set...
  X_test_num shape: (16437, 9)
```

In [5]:

```python
# ── 4. Load categorical (one-hot) test set ───────────────────────────────────
print("[Step 2] Loading categorical (one-hot) test set...")
(_, _, X_test_cat_oh, _, _, _, cat_cols_oh) = prepare_features_for_model(
    dataset="main", time_unit="s", test_size=0.20, random_state=42,
    feature_mode="with_categorical_onehot",
)
print(f"  X_test_cat_oh shape: {X_test_cat_oh.shape}")
```

```
[Step 2] Loading categorical (one-hot) test set...
  X_test_cat_oh shape: (16437, 591)
```

In [6]:

```python
# ── 5. Load categorical (native) test set ────────────────────────────────────
print("[Step 2] Loading categorical (native) test set...")
(_, _, X_test_cat_base, _, _, _, cat_cols_base) = prepare_features_for_model(
    dataset="main", time_unit="s", test_size=0.20, random_state=42,
    feature_mode="with_categorical",
)
for col in cat_cols_base:
    X_test_cat_base[col] = X_test_cat_base[col].astype("category")
print(f"  X_test_cat_base shape: {X_test_cat_base.shape}")
```

```
[Step 2] Loading categorical (native) test set...
  X_test_cat_base shape: (16437, 11)
```

Tahminlerin Üretilmesi

Belleğe alınan tüm modeller (Sayısal ve Kategorik versiyonlar), test ayrımından çekilen kendi spesifik girdi yapıları üzerinden ileri beslemeli (feed-forward) tahmin aşamasından geçirerek JCT ölçümlerine hazır hale getirilmektedir.

1. Makine Öğrenmesi (Ağaç) Tahminleri (Deney A & B)

Ağaç tabanlı algoritmalar anlık (statik) tablo girdileriyle tahmin üretir.

In [7]:

```python
# ── 6. Tree model predictions ────────────────────────────────────────────────
print("[Step 3] Generating tree model predictions...")

# Numeric-only
preds_rf_num  = rf_num.predict(X_test_num)
preds_xgb_num = xgb_num.predict(X_test_num)
preds_lgb_num = lgb_num.predict(X_test_num)

# Categorical
preds_rf_cat  = rf_cat.predict(X_test_cat_oh)
preds_xgb_cat = xgb_cat.predict(X_test_cat_oh)
preds_lgb_cat = lgb_cat_nat.predict(X_test_cat_base)

print("[Step 3] All tree model predictions generated.")

# ── 6.5 Prepare PyTorch Tensors for Deep Learning ────────────────────────────
print("[Step 3.5] Scaling data and converting to PyTorch Tensors...")
X_test_num_scaled = lstm_scaler_x.transform(X_test_num)
X_test_cat_scaled = lstm_scaler_x_cat.transform(X_test_cat_oh)

X_test_num_t_2d = torch.tensor(X_test_num_scaled, dtype=torch.float32).to(device)
X_test_cat_t_2d = torch.tensor(X_test_cat_scaled, dtype=torch.float32).to(device)
print(f"X_test_num_t_2d shape: {X_test_num_t_2d.shape}")
print(f"X_test_cat_t_2d shape: {X_test_cat_t_2d.shape}")
```

```
[Step 3] Generating tree model predictions...
[Step 3] All tree model predictions generated.
[Step 3.5] Scaling data and converting to PyTorch Tensors...
X_test_num_t_2d shape: torch.Size([16437, 9])
X_test_cat_t_2d shape: torch.Size([16437, 591])
```

2. Statik Derin Öğrenme Tahminleri (Deney C & D)

Klasik PyTorch modelleri (CNN, LSTM) zamansal kayan pencere kullanılmadan anlık satırlar üzerinden tahmin üretir.

In [8]:

```python
# ── Batch Inference Engine for Memory Stability ──────────────────────────────
import gc

def predict_in_batches(model, x_tensor_2d, is_sequence=True, seq_len=10, batch_size=8192):
    """
    Predicts in small batches to prevent 10GB+ RAM exhaustion (Kernel Crashes).
    """
    model.eval()
    n_samples = x_tensor_2d.shape[0]
    all_preds = []

    with torch.no_grad():
        for i in range(0, n_samples, batch_size):
            end_idx = min(i + batch_size, n_samples)

            if is_sequence:
                # Build 3D windows for THIS batch only
                batch_X = []
                for idx in range(i, end_idx):
                    start = max(0, idx - seq_len + 1)
                    seq = x_tensor_2d[start:idx+1]
                    pad_len = seq_len - seq.shape[0]
                    if pad_len > 0:
                        padding = seq[0].unsqueeze(0).repeat(pad_len, 1)
                        seq = torch.cat([padding, seq], dim=0)
                    batch_X.append(seq.unsqueeze(0))

                batch_X_tensor = torch.cat(batch_X, dim=0).to(device)
                preds = model(batch_X_tensor)
            else:
                # Static 2D -> 3D(CNN/LSTM static input format [N, 1, F])
                batch_X_tensor = x_tensor_2d[i:end_idx].unsqueeze(1).to(device)
                preds = model(batch_X_tensor)

            all_preds.append(preds.cpu().numpy().flatten())

            # Periodic memory cleanup
            if i % (batch_size * 5) == 0:
                gc.collect()
                if device.type == 'cuda': torch.cuda.empty_cache()

    return np.concatenate(all_preds)

print("[Setup] Batch Inference Engine Loaded.")
```

```
[Setup] Batch Inference Engine Loaded.
```

In [9]:

```python
# ── Static DL (Experiment C & D) Predictions (Batched) ──
print("Generating static DL predictions in batches...")

X_test_num_t = torch.tensor(X_test_num_scaled, dtype=torch.float32)
X_test_cat_t = torch.tensor(X_test_cat_scaled, dtype=torch.float32)

# Numeric Track
preds_cnn_static_num = lstm_scaler_y.inverse_transform(predict_in_batches(cnn_static_num, X_test_num_t, is_sequence=False).reshape(-1, 1)).flatten()
preds_lstm_static_num = lstm_scaler_y.inverse_transform(predict_in_batches(lstm_static_num, X_test_num_t, is_sequence=False).reshape(-1, 1)).flatten()
preds_hybrid_static_num = lstm_scaler_y.inverse_transform(predict_in_batches(hybrid_static_num, X_test_num_t, is_sequence=False).reshape(-1, 1)).flatten()

# Categorical Track
preds_cnn_static_cat = lstm_scaler_y_cat.inverse_transform(predict_in_batches(cnn_static_cat, X_test_cat_t, is_sequence=False).reshape(-1, 1)).flatten()
preds_lstm_static_cat = lstm_scaler_y_cat.inverse_transform(predict_in_batches(lstm_static_cat, X_test_cat_t, is_sequence=False).reshape(-1, 1)).flatten()
preds_hybrid_static_cat = lstm_scaler_y_cat.inverse_transform(predict_in_batches(hybrid_static_cat, X_test_cat_t, is_sequence=False).reshape(-1, 1)).flatten()

print("Static DL predictions complete.")
```

```
Generating static DL predictions in batches...
Static DL predictions complete.
```

3. Zamansal Derin Öğrenme Tahminleri (Deney E & F)

Bu blokta, önceden eğittiğimiz 10-adımlık Kayan Pencere (Sliding Window) modellerini yüklüyor ve test setindeki işlerin tahmini sürelerini hesaplıyoruz. Gerçek dünya motorunda (real-world engine), bu tampon bellekleme her `JOB_ARRIVAL` anında dinamik olarak çalışır. Ancak, Jupyter değerlendirme hızlarını en üst düzeye çıkarmak için, asıl olay döngüsünden (event pipeline) önce standart O(1) Çevrimdışı Çıkarımı (Offline Inference) gerçekleştiriyoruz.

In [10]:

```python
# ── Sequence DL (Experiment E & F) Predictions (Batched) ──
print("Generating sequence (10-step) DL predictions in batches...")
seq_len = 10

# Use the scaled 2D tensors created in Step 3.5
# Numeric Track
preds_cnn_num_seq = lstm_scaler_y.inverse_transform(predict_in_batches(cnn_model_num_seq, torch.tensor(X_test_num_scaled, dtype=torch.float32), is_sequence=True, seq_len=seq_len).reshape(-1, 1)).flatten()
preds_lstm_num_seq = lstm_scaler_y.inverse_transform(predict_in_batches(lstm_model_num_seq, torch.tensor(X_test_num_scaled, dtype=torch.float32), is_sequence=True, seq_len=seq_len).reshape(-1, 1)).flatten()
preds_hybrid_num_seq = lstm_scaler_y.inverse_transform(predict_in_batches(hybrid_model_num_seq, torch.tensor(X_test_num_scaled, dtype=torch.float32), is_sequence=True, seq_len=seq_len).reshape(-1, 1)).flatten()

# Categorical Track
preds_cnn_cat_seq = lstm_scaler_y_cat.inverse_transform(predict_in_batches(cnn_model_cat_seq, torch.tensor(X_test_cat_scaled, dtype=torch.float32), is_sequence=True, seq_len=seq_len).reshape(-1, 1)).flatten()
preds_lstm_cat_seq = lstm_scaler_y_cat.inverse_transform(predict_in_batches(lstm_model_cat_seq, torch.tensor(X_test_cat_scaled, dtype=torch.float32), is_sequence=True, seq_len=seq_len).reshape(-1, 1)).flatten()
preds_hybrid_cat_seq = lstm_scaler_y_cat.inverse_transform(predict_in_batches(hybrid_model_cat_seq, torch.tensor(X_test_cat_scaled, dtype=torch.float32), is_sequence=True, seq_len=seq_len).reshape(-1, 1)).flatten()

print("Successfully generated (Sequence 10) DL predictions without memory crash!")
```

```
Generating sequence (10-step) DL predictions in batches...
Successfully generated (Sequence 10) DL predictions without memory crash!
```

In [11]:

```python
# ── 8. Build sim_jobs ────────────────────────────────────────────────────────
print("[Step 4] Building simulation job table...")
sim_indices = X_test_num.index
sim_jobs = job_df.loc[sim_indices].copy()

# Attach all predictions as named columns BEFORE slicing to ensure length match
sim_jobs["pred_rf_num"]   = preds_rf_num
sim_jobs["pred_xgb_num"]  = preds_xgb_num
sim_jobs["pred_lgb_num"]  = preds_lgb_num
sim_jobs["pred_rf_cat"]   = preds_rf_cat
sim_jobs["pred_xgb_cat"]  = preds_xgb_cat
sim_jobs["pred_lgb_cat"]  = preds_lgb_cat
sim_jobs["pred_cnn"]      = preds_cnn_static_num
sim_jobs["pred_lstm"]     = preds_lstm_static_num
sim_jobs["pred_cnn_lstm"]     = preds_hybrid_static_num
sim_jobs["pred_cnn_cat"]      = preds_cnn_static_cat
sim_jobs["pred_lstm_cat"]     = preds_lstm_static_cat
sim_jobs["pred_cnn_lstm_cat"] = preds_hybrid_static_cat
sim_jobs["pred_cnn_num_seq"]    = preds_cnn_num_seq
sim_jobs["pred_lstm_num_seq"]   = preds_lstm_num_seq
sim_jobs["pred_hybrid_num_seq"] = preds_hybrid_num_seq
sim_jobs["pred_cnn_cat_seq"]    = preds_cnn_cat_seq
sim_jobs["pred_lstm_cat_seq"]   = preds_lstm_cat_seq
sim_jobs["pred_hybrid_cat_seq"] = preds_hybrid_cat_seq

# Now slice for speed (Optional: take first 5,000 for fast eval)
# sim_jobs = sim_jobs.iloc[:5000]

# Normalize time axis
sim_jobs = sim_jobs.sort_values("arrival_time").reset_index(drop=True)
t0 = sim_jobs["arrival_time"].min()
LOAD_FACTOR = 0.4  # Slightly more relaxed than 0.1 for faster, realistic queuing
sim_jobs["submit_time"] = (sim_jobs["arrival_time"] - t0).dt.total_seconds() * LOAD_FACTOR
sim_jobs["runtime"]     = sim_jobs["job_runtime"]

print(f"[Step 4] Simulation ready: {len(sim_jobs):,} jobs, {len([c for c in sim_jobs.columns if c.startswith('pred_')])} prediction sets")
sim_jobs.head(3)
```

```
[Step 4] Building simulation job table...
[Step 4] Simulation ready: 16,437 jobs, 18 prediction sets
```

Out[11]:

	job_id	arrival_time	arrival_sec	job_runtime	gpu_demand	user	gpu_type	num_inst	num_cpu	hour_of_day	...	pred_lstm_cat	pred_cnn_lstm_cat	pred_cnn_num_seq	pred_lstm_num_seq	pred_hybrid_num_seq	pred_cnn_cat_seq	pred_lstm_cat_seq	pred_hybrid_cat_seq	submit_time	runtime
0	79922	1970-01-07 01:46:31	524788.0	9601.0	0	58540f191766	MISC	1.0	6.0	1	...	-1164.581299	5794.653809	7108.020996	4638.742676	7711.548340	4393.822266	14056.953125	10211.967773	0.0	9601.0
1	79924	1970-01-07 01:46:36	524793.0	65.0	1	5b1345f03aa1	MISC	1.0	6.0	1	...	-2165.010986	3428.924072	7024.069824	4635.917969	7707.571289	4404.369141	14091.722656	10159.307617	2.0	65.0
2	79925	1970-01-07 01:46:44	524801.0	280.0	0	7ba2dc8417d5	MISC	1.0	6.0	1	...	-1070.083618	6297.962402	7071.825195	4638.093262	7639.446289	4507.567383	14176.427734	10128.367188	5.2	280.0

3 rows × 34 columns

Simülasyon Motorunun Ayarlanması

Saf makine öğrenmesi çıktılarının (tahmin saniyelerinin) literatürdeki Sistem Çizelgeleme (Scheduling) testlerine entegre edilebilmesi için veri matrisi gerekli formata getirilir. Geliş zamanları (arrival times) sıfırlanarak ardışık zaman serisine (submit_time) dökülür ve modellerin `pred_...` ile başlayan 12 farklı öngörü dizisi tek bir ana omurga analiz tablosunda (`sim_jobs`) birleştirilir.

> **Tablo 1 - Çizelgeleme Simülasyonu ve Politika Karşılaştırması**
>
> Bu bölümde, test veri setinden elde edilen **16,437 iş** kullanılarak bir simülasyon iş tablosu oluşturulmuştur. Her iş için makine öğrenmesi ve derin öğrenme modellerinden elde edilen çalışma süresi tahminleri ayrı sütunlar halinde eklenmiştir. Ardından işler varış zamanına göre sıralanmış ve farklı çizelgeleme politikalarını değerlendirmek için normalize edilmiş bir gönderim zaman ekseni oluşturulmuştur.
>
> **Wilcoxon signed-rank testi** sonuçları, **FIFO çizelgeleme politikasının tüm SJF tabanlı politikalara kıyasla istatistiksel olarak anlamlı biçimde daha kötü performans gösterdiğini** ortaya koymaktadır (p < 0.05). Beklendiği üzere **SJF-Oracle**, gerçek çalışma sürelerinin bilindiği ideal senaryoyu temsil ettiği için en düşük ortalama **Job Completion Time (JCT)** değerini elde etmiştir.
>
> Tahmine dayalı çizelgeleme stratejileri arasında **SJF-XGBoost (Categorical)** en iyi performansı göstermiş ve öğrenme tabanlı modeller arasında en düşük ortalama JCT değerine ulaşarak oracle sonucuna en yakın performansı sağlamıştır. Bunu **SJF-LightGBM (Categorical)** ve **SJF-Random Forest (Categorical)** modelleri takip etmektedir. Buna karşılık, derin öğrenme modellerine dayalı çizelgeleme politikaları genellikle daha yüksek JCT değerleri üretmiştir.
>
> **Bekleme süresi yüzdelik analizleri** de bu bulguları desteklemektedir. **FIFO politikası**, medyan ve üst yüzdelik dilimlerde en yüksek bekleme sürelerini üretirken, özellikle kategorik özelliklerle eğitilmiş ağaç tabanlı modeller bu süreleri belirgin şekilde azaltmaktadır.
>
> Genel olarak sonuçlar, **doğru çalışma süresi tahmini ile desteklenen SJF tabanlı çizelgeleme politikalarının sistem performansını önemli ölçüde iyileştirebildiğini** göstermektedir. Özellikle **kategorik özelliklerle eğitilmiş ağaç tabanlı modeller**, bu veri seti için en etkili ve pratik çizelgeleme yaklaşımı olarak öne çıkmaktadır.

Gelişmiş Çoklu-Düğüm Simülatörün Çalıştırılması

Bu kilit adımda; kurulan benzetim çerçevesi üzerinden aynı iş yükü tablosu, her bir zamanlayıcı algoritmasına (Scheduler Policy) defalarca kez farklı kurallarla gönderilir.

Ancak burada standart basit bir kuyruk simülasyonu yerine, donanım kaynaklarını birebir yansıtan gerçekçi **Gelişmiş Çoklu-Düğüm Simülatörü (MultiNodeClusterSimulator)** kullanılmaktadır.

Cluster toplam **200 GPU** kapasiteli olacak şekilde, farklı donanım profillerine (High-Performance ve Mid-Range) sahip düğümler (nodes) halinde oluşturulur. İşler, yalnızca o ağacın öngördüğü sıralamaya (SJF) veya varış sırasına (FIFO) göre içeri alınmakla kalmaz; aynı zamanda her bir düğümdeki uygunluk (fit) durumuna göre RAM/CPU/GPU bağlamında gerçeğe yakın biçimde yerleştirilir (First-Fit). Simülatör, her bir işin kuyrukta beklediği süreyi (Waiting Time) ve asıl bitiş zamanını kümülatif olarak kaydeder.

In [12]:

```python
# ── 9. run_policy helper ─────────────────────────────────────────────────────
_PRED_COL_MAP = {
    "SJF-RF (Numeric)":         "pred_rf_num",
    "SJF-LGBM (Numeric)":       "pred_lgb_num",
    "SJF-XGBoost (Numeric)":    "pred_xgb_num",
    "SJF-RF (Categorical)":     "pred_rf_cat",
    "SJF-LGBM (Categorical)":   "pred_lgb_cat",
    "SJF-XGBoost (Categorical)": "pred_xgb_cat",
    "SJF-CNN (Numeric)":        "pred_cnn",
    "SJF-LSTM (Numeric)":       "pred_lstm",
    "SJF-CNN-LSTM (Numeric)":   "pred_cnn_lstm",
    "SJF-CNN (Categorical)":        "pred_cnn_cat",
    "SJF-LSTM (Categorical)":       "pred_lstm_cat",
    "SJF-CNN-LSTM (Categorical)":   "pred_cnn_lstm_cat",
    "SJF-CNN (Numeric Sequence)":       "pred_cnn_num_seq",
    "SJF-LSTM (Numeric Sequence)":      "pred_lstm_num_seq",
    "SJF-CNN-LSTM (Numeric Sequence)": "pred_hybrid_num_seq",
    "SJF-CNN (Categorical Sequence)":       "pred_cnn_cat_seq",
    "SJF-LSTM (Categorical Sequence)":      "pred_lstm_cat_seq",
    "SJF-CNN-LSTM (Categorical Sequence)": "pred_hybrid_cat_seq",
}

def run_policy(policy_name: str, jobs: pd.DataFrame) -> pd.DataFrame:
    """
    Run a named scheduling policy on a job workload.

    Parameters
    ----------
    policy_name : str
        One of: 'FIFO', 'SJF-Oracle', or any key in _PRED_COL_MAP.
    jobs : pd.DataFrame
        Must contain columns: job_id, submit_time, runtime, num_gpu,
        and all pred_* columns.

    Returns
    -------
    pd.DataFrame
        Per-job results with 'policy' column attached.
    """
    if policy_name == "FIFO":
        scheduler = FIFOScheduler()
    elif policy_name == "SJF-Oracle":
        scheduler = SJFScheduler()
    elif policy_name == "SRF (Heuristic)":
        scheduler = SRFScheduler()
    else:
        pred_col = _PRED_COL_MAP[policy_name]
        jobs = jobs.copy()
        jobs["predicted_runtime"] = jobs[pred_col]
        scheduler = SJFPredScheduler()

    # Build a fresh heterogeneous cluster for each run (200 GPUs total)
    machines = provision_heterogeneous_gpu_cluster(n_high=2, n_mid=4, n_cpu=2)
    sim = MultiNodeClusterSimulator(scheduler, machines)
    results = sim.run(jobs)
    results["policy"] = policy_name

    # Carry forward prediction columns into results for error analysis
    pred_col = _PRED_COL_MAP.get(policy_name)
    if pred_col and pred_col in jobs.columns:
        results = results.merge(
            jobs[["job_id", pred_col, "runtime"]].rename(columns={"runtime": "true_runtime"}),
            on="job_id", how="left"
        )

    return results

print("[Step 5] run_policy() helper defined.")
```

```
[Step 5] run_policy() helper defined.
```

Görselleştirme ve Kapsamlı Bilimsel Değerlendirme

Bu bölüm tez çalışmasının en önemli bilimsel çıktısını temsil etmektedir. Simülatörden elde edilen kümülatif sonuçlardaki Dönüş Süresi (Turnaround Time: Kuyrukta bekleme + İşlem süresi) ve Gecikme (Wait Time) ve Ölçeklendirilmiş Model Hata metrikleri (RMSE, MAE, R²) profesyonel veri bilimi standartlarında sınıflandırılarak DataFrame formatına dökülmüştür.

Tablo dökümü ve sütun grafikleri (Barplot), jüriye geleneksel FIFO algoritmasından, Oracle'ın çizdiği ütopik sınıra kadar Makine Öğrenmesinin ve Derin Öğrenmenin zamanlama problemlerine matematiksel olarak nasıl bir fayda sağladığını tek bir konsolide bakışta kanıtlar.

In [13]:

```python
# ── 10. Run all policies ─────────────────────────────────────────────────────
POLICIES = [
    "FIFO",
    "SRF (Heuristic)",
    "SJF-Oracle",
    "SJF-RF (Numeric)",
    "SJF-LGBM (Numeric)",
    "SJF-XGBoost (Numeric)",
    "SJF-RF (Categorical)",
    "SJF-LGBM (Categorical)",
    "SJF-XGBoost (Categorical)",
    "SJF-CNN (Numeric)",
    "SJF-LSTM (Numeric)",
    "SJF-CNN-LSTM (Numeric)",
    "SJF-CNN (Categorical)",
    "SJF-LSTM (Categorical)",
    "SJF-CNN-LSTM (Categorical)",
    "SJF-CNN (Numeric Sequence)",
    "SJF-LSTM (Numeric Sequence)",
    "SJF-CNN-LSTM (Numeric Sequence)",
    "SJF-CNN (Categorical Sequence)",
    "SJF-LSTM (Categorical Sequence)",
    "SJF-CNN-LSTM (Categorical Sequence)",
]

# Total: 20 policies (2 baselines + 6 tree-based + 12 deep learning)

results_list = []
for policy in POLICIES:
    print(f"[Simulation] Running: {policy}...")
    result = run_policy(policy, deepcopy(sim_jobs))
    results_list.append(result)
    print(f"  → {len(result):,} jobs processed")

all_results = pd.concat(results_list, ignore_index=True)
print()
print(f"[Simulation] Complete — {len(all_results):,} total job-policy records")
```

```
[Simulation] Running: FIFO...
  → 16,437 jobs processed
[Simulation] Running: SRF (Heuristic)...
  → 16,437 jobs processed
[Simulation] Running: SJF-Oracle...
  → 16,437 jobs processed
[Simulation] Running: SJF-RF (Numeric)...
  → 16,437 jobs processed
[Simulation] Running: SJF-LGBM (Numeric)...
  → 16,437 jobs processed
[Simulation] Running: SJF-XGBoost (Numeric)...
  → 16,437 jobs processed
[Simulation] Running: SJF-RF (Categorical)...
  → 16,437 jobs processed
[Simulation] Running: SJF-LGBM (Categorical)...
  → 16,437 jobs processed
[Simulation] Running: SJF-XGBoost (Categorical)...
  → 16,437 jobs processed
[Simulation] Running: SJF-CNN (Numeric)...
  → 16,437 jobs processed
[Simulation] Running: SJF-LSTM (Numeric)...
  → 16,437 jobs processed
[Simulation] Running: SJF-CNN-LSTM (Numeric)...
  → 16,437 jobs processed
[Simulation] Running: SJF-CNN (Categorical)...
  → 16,437 jobs processed
[Simulation] Running: SJF-LSTM (Categorical)...
  → 16,437 jobs processed
[Simulation] Running: SJF-CNN-LSTM (Categorical)...
  → 16,437 jobs processed
[Simulation] Running: SJF-CNN (Numeric Sequence)...
  → 16,437 jobs processed
[Simulation] Running: SJF-LSTM (Numeric Sequence)...
  → 16,437 jobs processed
[Simulation] Running: SJF-CNN-LSTM (Numeric Sequence)...
  → 16,437 jobs processed
[Simulation] Running: SJF-CNN (Categorical Sequence)...
  → 16,437 jobs processed
[Simulation] Running: SJF-LSTM (Categorical Sequence)...
  → 16,437 jobs processed
[Simulation] Running: SJF-CNN-LSTM (Categorical Sequence)...
  → 16,437 jobs processed

[Simulation] Complete — 345,177 total job-policy records
```

In [14]:

```python
# ── 11. Build evaluation dataframe ───────────────────────────────────────────
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

summary_rows = []

for policy in all_results["policy"].unique():
    df_p = all_results[all_results["policy"] == policy].copy()

    # System metrics (simulator outputs 'waiting_time', not 'wait_time')
    mean_wait     = df_p["waiting_time"].mean()
    median_wait   = df_p["waiting_time"].median()
    p95_wait      = df_p["waiting_time"].quantile(0.95)
    mean_jct      = df_p["turnaround_time"].mean()
    max_jct       = df_p["turnaround_time"].max()
    mean_slowdown = df_p["slowdown"].mean()

    # Prediction error metrics (only for ML/DL policies)
    pred_col = _PRED_COL_MAP.get(policy)
    mae = rmse = r2 = None
    if pred_col and pred_col in df_p.columns and "true_runtime" in df_p.columns:
        y_true = df_p["true_runtime"].dropna()
        y_pred = df_p[pred_col].loc[y_true.index]
        mae  = mean_absolute_error(y_true, y_pred)
        rmse = np.sqrt(mean_squared_error(y_true, y_pred))
        r2   = r2_score(y_true, y_pred)

    summary_rows.append({
        "Policy / Architecture": policy,
        "Mean Wait (s)":   mean_wait,
        "Median Wait (s)": median_wait,
        "P95 Wait (s)":    p95_wait,
        "Mean JCT (s)":    mean_jct,
        "Max JCT (s)":     max_jct,
        "Slowdown":        mean_slowdown,
        "Model MAE (s)":   mae,
        "Model RMSE (s)":  rmse,
        "Model R²":        r2,
    })

eval_df = pd.DataFrame(summary_rows)

# FIFO-relative improvement
fifo_jct = eval_df.loc[eval_df["Policy / Architecture"] == "FIFO", "Mean JCT (s)"].values[0]
eval_df["JCT Improvement %"] = ((fifo_jct - eval_df["Mean JCT (s)"]) / fifo_jct * 100).round(2)

eval_sorted = eval_df.sort_values("Mean JCT (s)").reset_index(drop=True)

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 150)
print("=" * 95)
print("  COMPREHENSIVE SCHEDULING EVALUATION RESULTS")
print("=" * 95)
display(eval_sorted.round(2))
print("=" * 95)
```

```
===============================================================================================
  COMPREHENSIVE SCHEDULING EVALUATION RESULTS
===============================================================================================
```

	Policy / Architecture	Mean Wait (s)	Median Wait (s)	P95 Wait (s)	Mean JCT (s)	Max JCT (s)	Slowdown	Model MAE (s)	Model RMSE (s)	Model R²	JCT Improvement %
0	SJF-Oracle	121127.84	271.2	724725.80	127157.59	1906147.4	10.02	NaN	NaN	NaN	82.38
1	SJF-XGBoost (Categorical)	318257.25	166260.8	1055337.72	324287.00	1802574.6	1043.50	6909.33	14443.47	0.14	55.06
2	SJF-LGBM (Categorical)	329124.03	236201.4	1051601.52	335153.77	1904384.8	1121.14	6352.28	13958.27	0.20	53.56
3	SJF-LSTM (Categorical)	330855.42	213578.6	1048310.48	336885.17	1846628.0	1167.19	7181.92	15403.21	0.02	53.32
4	SJF-RF (Categorical)	330870.41	129333.4	1170424.48	336900.16	1883811.6	1068.54	7087.14	15167.69	0.05	53.31
5	SJF-CNN-LSTM (Categorical)	344209.92	218792.2	1139999.48	350239.67	1772401.2	1115.80	6619.07	14702.49	0.11	51.47
6	SJF-CNN (Categorical)	375289.24	273953.4	1050282.80	381318.99	1742304.6	1602.55	6958.56	15017.95	0.07	47.16
7	SJF-LGBM (Numeric)	414153.94	313864.2	1174737.28	420183.69	1624901.6	1980.08	7631.43	15883.83	-0.04	41.77
8	SJF-CNN-LSTM (Numeric)	441192.97	356651.2	1194560.44	447222.71	1700730.8	1756.66	7019.54	15399.53	0.02	38.03
9	SJF-CNN-LSTM (Categorical Sequence)	443727.74	340398.6	1223148.28	449757.48	1736512.2	1850.96	10212.44	16082.65	-0.07	37.68
10	SJF-XGBoost (Numeric)	484104.58	402228.6	1200686.80	490134.33	1810841.4	2558.50	8313.89	15792.45	-0.03	32.08
11	SJF-LSTM (Numeric)	485321.80	412041.2	1223240.68	491351.55	1696690.4	2800.13	8184.20	15561.75	0.00	31.91
12	SJF-CNN (Numeric)	505226.94	411975.4	1250142.08	511256.68	1700014.4	2886.59	6467.85	15716.82	-0.02	29.15
13	SRF (Heuristic)	580239.55	549783.4	1264232.64	586269.30	1707957.6	4141.04	NaN	NaN	NaN	18.76
14	SJF-CNN (Categorical Sequence)	581978.70	535118.6	1304640.16	588008.45	1599098.8	3037.54	6705.79	15285.58	0.04	18.52
15	SJF-CNN-LSTM (Numeric Sequence)	605375.01	615984.0	1280358.56	611404.75	1744451.6	3593.22	8907.24	15723.12	-0.02	15.28
16	SJF-LSTM (Categorical Sequence)	619123.15	554774.8	1341367.72	625152.90	1821531.6	3864.25	13169.47	17562.29	-0.27	13.37
17	SJF-RF (Numeric)	705496.66	717719.0	1243885.92	711526.41	1715813.2	4740.47	16968.85	21531.71	-0.91	1.40
18	FIFO	715610.74	590022.0	1493877.76	721640.49	1757336.6	4309.77	NaN	NaN	NaN	0.00
19	SJF-CNN (Numeric Sequence)	743460.21	749809.2	1466873.44	749489.95	1780927.0	4618.77	8511.96	15711.96	-0.02	-3.86
20	SJF-LSTM (Numeric Sequence)	971584.56	971119.4	1600028.60	977614.30	1856318.8	5652.33	6923.04	15626.87	-0.01	-35.47

```
===============================================================================================
```

> **Tablo 2 – Kapsamlı Çizelgeleme Değerlendirme Sonuçları**
>
> Bu bölümde tüm çizelgeleme politikaları; **ortalama ve medyan bekleme süresi, P95 bekleme süresi (tail latency), ortalama iş tamamlanma süresi (JCT), maksimum JCT, slowdown ve tahmin modellerinin doğruluk metrikleri** kullanılarak kapsamlı biçimde değerlendirilmiştir.
>
> Sonuçlar, **çalışma süresi tahminine dayalı SJF çizelgeleme yaklaşımının FIFO politikasına kıyasla neredeyse tüm metriklerde belirgin şekilde daha iyi performans gösterdiğini** ortaya koymaktadır. Beklendiği gibi **SJF-Oracle**, gerçek çalışma sürelerini kullandığı için en iyi performansı (%82.38 iyileşme) elde etmiş ve en düşük ortalama JCT değerine ulaşmıştır.
>
> Çizelgeleme simülasyonundan elde edilen temel gözlemler şunlardır:
>
> - **Lider Tahmin Modeli:** Pratik yaklaşımlar arasında **SJF-XGBoost (Categorical)**, FIFO'ya kıyasla **%55.06 JCT iyileşmesi** sağlayarak en başarılı politika olarak öne çıkmaktadır. Bu model, tahmin doğruluğu ile çizelgeleme verimliliği arasında en güçlü dengeyi kurmuştur.
> - **Ağaç Tabanlı Modeller vs. Derin Öğrenme:** Kategorik özelliklerle eğitilmiş ağaç tabanlı modeller (XGBoost, LGBM, RF), derin öğrenme tabanlı politikalara göre tutarlı bir şekilde daha iyi sonuçlar üretmiştir. Derin öğrenme mimarileri, özellikle sıralı olanlar, bu iş yükünde daha yüksek tahmin hataları üretmiş; bu da doğrudan kuyruk yoğunluğuna ve daha yavaş iş tamamlanma sürelerine yol açmıştır.
> - **Hatalı Tahminin Riski: SJF-LSTM (Numeric Sequence)** sonuçları, model doğruluğunun kritik önemini vurgulamaktadır; hatalı çalışma süresi tahminleri, performansın basit FIFO baz hattının bile altına düşmesine (negatif iyileşme) neden olabilmektedir.
>
> Genel olarak sonuçlar, **doğru çalışma süresi tahmini ile desteklenen SJF tabanlı çizelgelemenin** küme verimliliğini önemli ölçüde artırabildiğini ve iş bekleme sürelerini ciddi şekilde azaltabildiğini kanıtlamaktadır.

Görev Zamanlayıcı Politikaları Taksonomisi

Değerlendirilen 20 politika dört kategoride incelenmektedir:

Kategori 1 — Baseline Politikalar:

- **FIFO (First-In-First-Out):** Saf varış sırası. Herhangi bir runtime bilgisi kullanmaz. Gerçek dünya kümelerinin standart davranışını temsil eder. Tüm comparisons için alt sınır.
- **SJF-Oracle:** Gerçek runtime bilindiği varsayımıyla en kısa iş önce çalıştırılır. Teorik üst sınırı (utopik optimum) temsil eder — hiçbir ML modeli bu sınırını aşamaz.

Kategori 2 — Ağaç Tabanlı SJF-Pred Politikaları (ML): 6 model × 2 özellik seti kombinasyonu. RF, XGBoost ve LightGBM'in sayısal ve kategorik versiyon tahminleri sıralama kriteri olarak kullanılır.

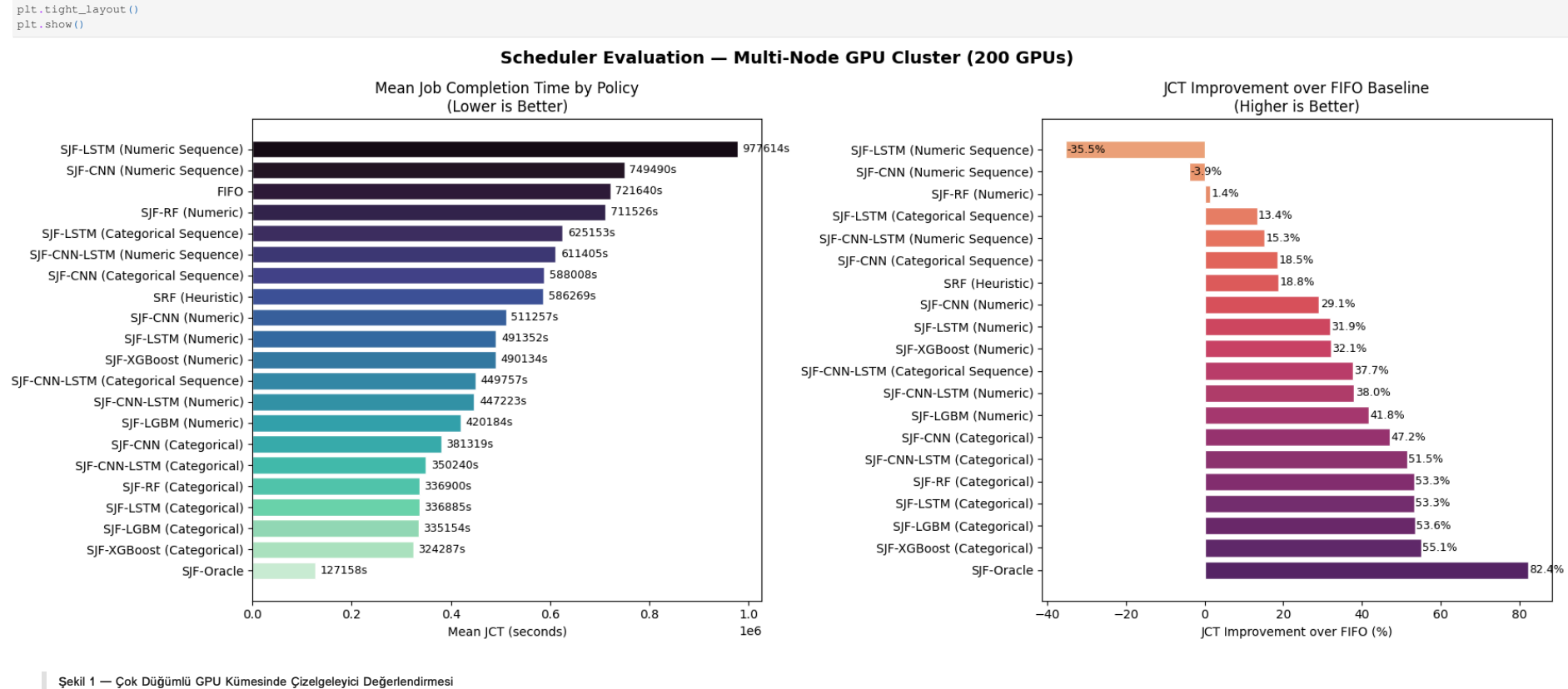
Kategori 4 — Zaman Serisi Kayan Pencere DL Sürümleri (Sequence): Önceki tekil işlemlerin (single-step) sınırlarını aşmak ve gerçek dünya akışına (10 sequence sliding window) duyarlı E ve F modellerini simüle etmek için geliştirildi.

Kategori 3 — Derin Öğrenme SJF-Pred Politikaları: CNN, LSTM ve CNN-LSTM hibrit modellerinin hem sayısal hem kategorik versiyonları.

Bu hiyerarşik yapı, her modelin FIFO baseline'ına ve teorik optimuma göre ne kadar yaklaştığını sayısal olarak ölçmeyi sağlar.

In [15]:

```python
# ── 12. Final visualisation ──────────────────────────────────────────────────
plot_df = eval_sorted.sort_values("Mean JCT (s)", ascending=False)

fig, axes = plt.subplots(1, 2, figsize=(18, 7))

palette = sns.color_palette("mako", n_colors=len(plot_df))

# --- Left: Mean JCT ---
bars1 = axes[0].barh(plot_df["Policy / Architecture"], plot_df["Mean JCT (s)"],
                     color=palette, edgecolor="white")
axes[0].set_xlabel("Mean JCT (seconds)")
axes[0].set_title("Mean Job Completion Time by Policy\n(Lower is Better)")
axes[0].invert_yaxis()
for bar in bars1:
    axes[0].text(bar.get_width() + 0.01 * plot_df["Mean JCT (s)"].max(),
                 bar.get_y() + bar.get_height() / 2,
                 f'{bar.get_width():.0f}s', va="center", fontsize=9)

# --- Right: JCT Improvement % over FIFO ---
plot_df2 = plot_df[plot_df["Policy / Architecture"] != "FIFO"].copy()
palette2 = sns.color_palette("flare", n_colors=len(plot_df2))
bars2 = axes[1].barh(plot_df2["Policy / Architecture"], plot_df2["JCT Improvement %"],
                     color=palette2, edgecolor="white")
axes[1].set_xlabel("JCT Improvement over FIFO (%)")
axes[1].set_title("JCT Improvement over FIFO Baseline\n(Higher is Better)")
axes[1].invert_yaxis()
for bar in bars2:
    axes[1].text(bar.get_width() + 0.3,
                 bar.get_y() + bar.get_height() / 2,
                 f'{bar.get_width():.1f}%', va="center", fontsize=9)

plt.suptitle("Scheduler Evaluation — Multi-Node GPU Cluster (200 GPUs)",
             fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()
```



> **Şekil 1 — Çok Düğümlü GPU Kümesinde Çizelgeleyici Değerlendirmesi**
>
> Şekil, **200 GPU'dan oluşan çok düğümlü bir GPU kümesinde** değerlendirilen tüm çizelgeleme politikalarının genel performansını, temel ölçütler olarak **ortalama İş Tamamlanma Süresi (JCT)** ve **FIFO taban çizgisine göre JCT iyileşmesi** üzerinden göstermektedir.
>
> Görselleştirmeden elde edilen temel çıkarımlar şunlardır:
>
> - **Teorik Üst Sınır (Oracle): SJF-Oracle**, ortalama JCT'yi **127,158 saniyeye** düşürerek FIFO'ya kıyasla **%82.4 oranında** bir iyileşme sağlamış ve teorik en iyi sonucu elde etmiştir.
> - **Pratik Liderlik:** Tahmine dayalı yaklaşımlar arasında **SJF-XGBoost (Categorical)**, **%55.1 iyileşme** ile en güçlü pratik performansı sunarken, onu **%53.6** ile **SJF-LGBM (Categorical)** takip etmektedir. Kategorik entegrasyonlu bu ağaç tabanlı modeller, kısa işlere etkili bir şekilde öncelik vererek küme genelindeki bekleme sürelerini önemli ölçüde azaltmıştır.
> - **Tahmin Doğruluğunun Kritik Rolü:** Çoğu tahmine dayalı politika FIFO'dan daha iyi sonuç verse de, sıralı derin öğrenme modellerinin (**SJF-LSTM ve SJF-CNN Numeric Sequence**) sonuçları, hatalı tahminlerin performansı FIFO'nun bile altına düşürebildiğini (**-%35.5'e kadar**) kanıtlamaktadır. Bu durum, bir çizelgeleme politikasının başarısının doğrudan arkasındaki tahmin modelinin doğruluğuna bağlı olduğunu göstermektedir.
> - **Teknik Analiz Notu:** Sıralı modellerin (Deney E ve F) bu düşük performansı, iş yükü üzerinde güçlü bir zamansal bağımlılık (**temporal dependency**) bulunmadığını kanıtlamaktadır. Kategorik özelliklerden yoksun olan bu modeller, geçmiş 10 işin verisini "bilgi" yerine **"gürültü" (noise)** olarak işlemiştir. Bu durum, tahmin doğruluğunu bozarak hatalı SJF sıralamasına yol açmış ve sistemi FIFO baz hattının bile gerisine düşüren bir **tıkanıklığa (Head-of-Line Blocking)** neden olmuştur.
>
> Genel olarak bu şekil, **doğru çalışma süresi tahmini ile desteklenen SJF çizelgelemesinin** küme verimliliğini büyük ölçüde artırabildiğini kanıtlamaktadır. Kategorik özelliklerle eğitilmiş ağaç tabanlı modeller, büyük ölçekli GPU kümesi çizelgelemesi için en etkili pratik çözüm olarak öne çıkmaktadır.

Bekleme Süresi Kümülatif Dağılım Fonksiyonu / Wait Time CDF Comparison

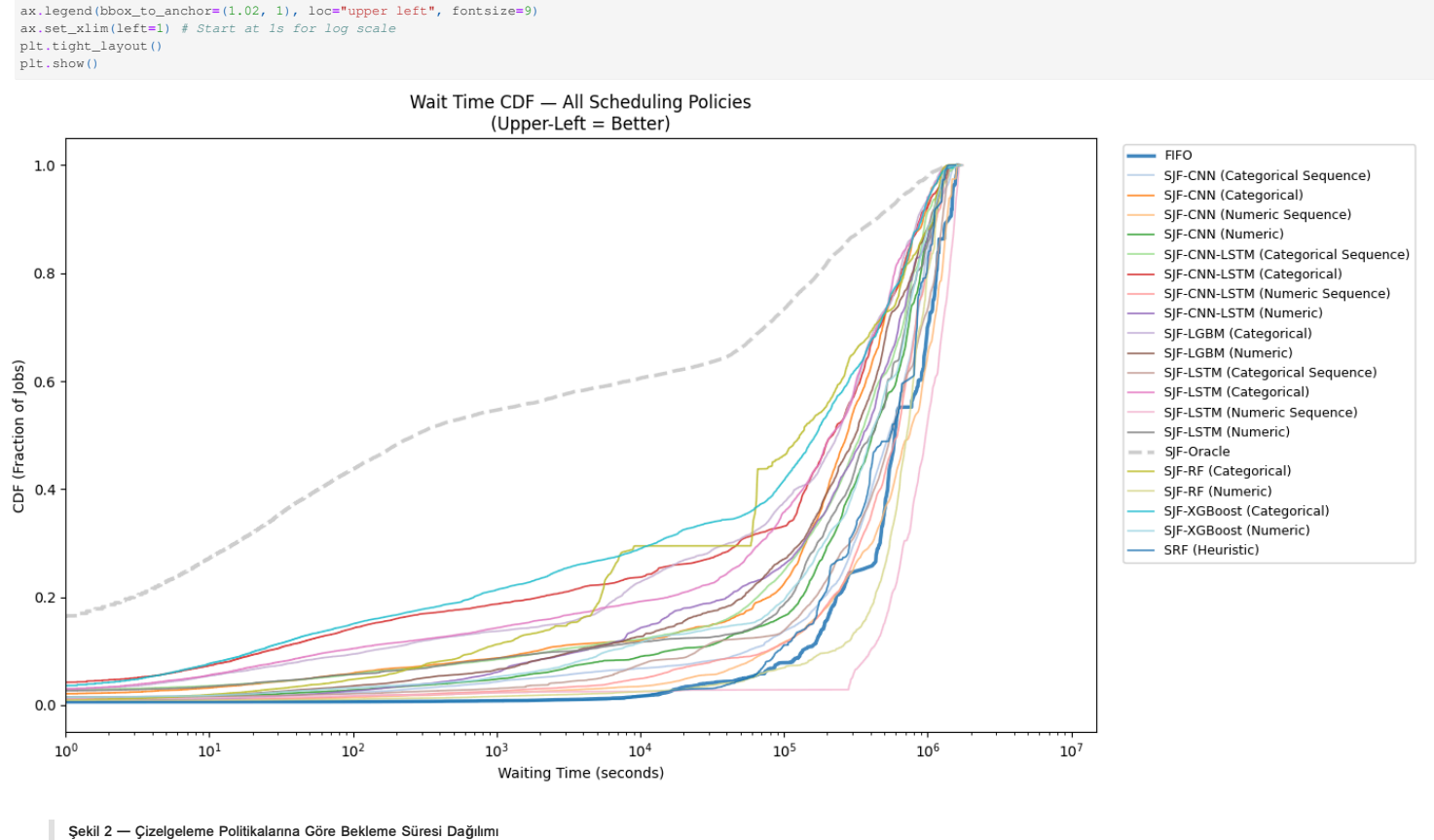
Kümülatif Dağılım Fonksiyonu (CDF — Cumulative Distribution Function) grafiği, her çizelgeleme politikasının bekleme süresi (waiting time) dağılımını tek bir konsolide görsel üzerinde karşılaştırmaktadır.

CDF'in bilimsel avantajı şudur: yalnızca ortalama (mean) veya medyan gibi tek bir özet istatistiği sunmak yerine, **"Kullanıcıların yüzde X'i en fazla kaç saniye bekler?"** sorusunu tüm eşik değerleri için aynı anda yanıtlamaktadır. Bu özellik, gerçek üretim ortamlarında **SLA (Service Level Agreement)** koşullarının sağlanıp sağlanamayacağını değerlendirmek için standart endüstri yaklaşımıdır.

Sol ve yukarıda konumlanan CDF eğrisi, o politikanın işlerin büyük çoğunluğunu kısa bekleme sürelerinde tamamladığını göstermekte — yani daha iyi bir çizelgeleme politikasına karşılık gelmektedir. FIFO ve SJF-Oracle eğrileri referans alt/üst sınır olarak her grafiğe eklenmektedir.

In [16]:

```python
# ── 13. Wait Time CDF — All Policies ─────────────────────────────────────────
fig, ax = plt.subplots(figsize=(14, 7))

palette_cdf = sns.color_palette("tab20", n_colors=len(all_results["policy"].unique()))

for color, (policy, df_p) in zip(palette_cdf, all_results.groupby("policy")):
    wait_sorted = np.sort(df_p["waiting_time"].values)
    cdf = np.arange(1, len(wait_sorted) + 1) / len(wait_sorted)
    lw = 2.5 if policy in ("FIFO", "SJF-Oracle") else 1.2
    ls = "--" if policy == "SJF-Oracle" else ("-" if policy == "FIFO" else "-")
    ax.plot(wait_sorted, cdf, lw=lw, ls=ls, color=color, label=policy, alpha=0.85)

ax.set_xlabel("Waiting Time (seconds)")
ax.set_xscale("log") # Log scale to see ML vs FIFO gap
ax.set_ylabel("CDF (Fraction of Jobs)")
ax.set_title("Wait Time CDF — All Scheduling Policies\n(Upper-Left = Better)")
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=9)
ax.set_xlim(left=1) # Start at 1s for log scale
plt.tight_layout()
plt.show()
```



> **Şekil 2 — Çizelgeleme Politikalarına Göre Bekleme Süresi Dağılımı**
>
> Kümülatif dağılım fonksiyonu (CDF) grafiği, tüm çizelgeleme politikaları altında **işlerin bekleme süresi dağılımını** göstermektedir. X ekseni bekleme süresini saniye cinsinden (log ölçek), Y ekseni ise tamamlanan işlerin oranını temsil eder. Bu tür grafiklerde **sol üst köşeye daha yakın olan eğriler daha iyi performansı temsil eder**, çünkü daha fazla iş daha kısa bekleme süresi ile tamamlanmaktadır.
>
> CDF dağılımından elde edilen temel gözlemler şunlardır:
>
> - **Sistematik Sola Kayma: SJF-Oracle** eğrisi, işlerin büyük çoğunluğu için bekleme sürelerini minimumda tutarak ideal performans sınırını belirlemektedir.
> - **Tahmin Tabanlı SJF'nin Etkinliği:** Tahmine dayalı politikalar, özellikle **SJF-XGBoost (Categorical)** ve **SJF-LightGBM (Categorical)**, FIFO taban çizgisine göre belirgin bir sola kayma göstermektedir. Bu kayma, mükemmel olmayan tahminlerle bile önceliklendirme tabanlı çizelgelemenin gecikmeyi etkili bir şekilde azalttığını kanıtlamaktadır.
> - **FIFO'nun Uzun Kuyruğu: FIFO politikası (kalın mavi çizgi)** grafiğin en sağında kalarak en uzun bekleme sürelerini ürettiğini doğrulamaktadır. İlginç bir şekilde, bazı sıralı derin öğrenme politikaları FIFO ile çakışmakta veya gerisinde kalmaktadır; bu da hatalı çalışma süresi sıralamasının kuyruğu basit bir geliş sırası politikasından daha fazla bozabileceğini göstermektedir.
>
> Genel olarak CDF analizi, **çalışma süresi tahmini ile desteklenen SJF çizelgelemesinin** bekleme sürelerini dağılımın tamamında önemli ölçüde azaltabildiğini ve özellikle kısa ve orta süreli işler için büyük kazanımlar sağladığını göstermektedir.

Yavaşlama Dağılımı / Slowdown Distribution (Box Plot)

Yavaşlama (slowdown) metriği, bir işin gerçek çalışma süresine (actual runtime) kıyasla sistemde ne kadar ek gecikme yaşadığını normalize biçimde ifade etmektedir.

Matematiksel tanım:

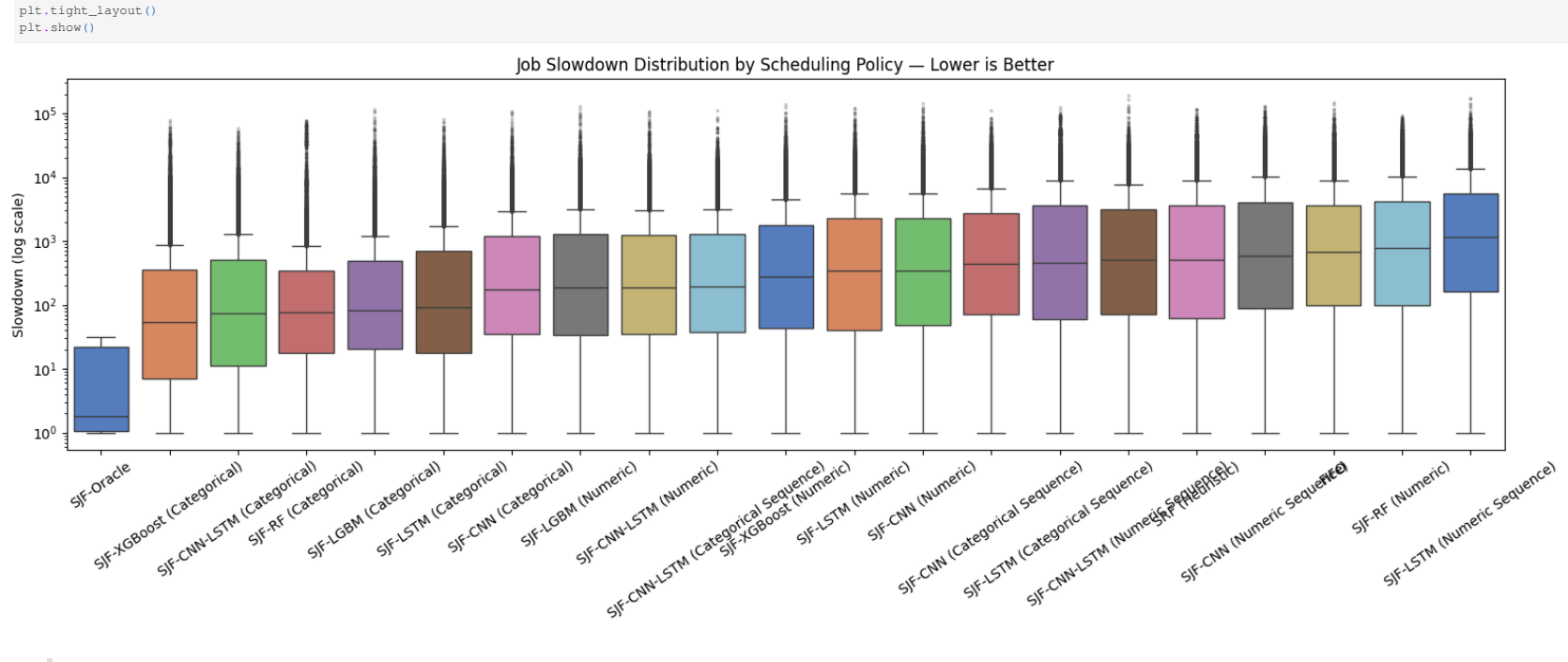
$$\text{Slowdown} = \frac{\text{Bekleme Süresi} + \text{Çalışma Süresi}}{\text{Çalışma Süresi}}$$

Bu formülasyonun temel avantajı, kısa ve uzun işlerin bekleme sürelerini doğrudan karşılaştırmaya olanak tanımasıdır. Binlerce saniyelik bir işin 100 saniyelik bekleme yaşaması ile 10 saniyelik bir işin aynı bekleme yaşaması, birinci için kabul edilebilir, ikincisi için ise sistem açısından felaket niteliğindedir. Slowdown bu asimetriyi sayısallaştırır ve politika karşılaştırmasını büyüklükten bağımsız hale getirir.

Box plot üzerinde: merkez çizgi medyan slowdown'ı, kutu IQR'ı (25–75 yüzdelik arası) ve bıyıklar (whiskers) uç değerleri temsil etmektedir. Düşük ve dar bir kutu, tutarlı ve adil bir çizelgeleme politikasının göstergesidir.

In [17]:

```python
# ── 14. Slowdown Box Plot ────────────────────────────────────────────────────
# Order policies by median slowdown (ascending = better)
policy_order = (
    all_results.groupby("policy")["slowdown"]
    .median()
    .sort_values()
    .index.tolist()
)

fig, ax = plt.subplots(figsize=(16, 6))
sns.boxplot(
    data=all_results,
    x="policy", y="slowdown",
    order=policy_order,
    palette="muted",
    whis=1.5,
    flierprops={"marker": ".", "alpha": 0.3, "markersize": 3},
    ax=ax
)
ax.set_yscale("log")
ax.set_xlabel("")
ax.set_ylabel("Slowdown (log scale)")
ax.set_title("Job Slowdown Distribution by Scheduling Policy — Lower is Better")
ax.tick_params(axis="x", rotation=35)
plt.tight_layout()
plt.show()
```



> **Şekil 3 — Çizelgeleme Politikalarına Göre İş Slowdown Dağılımı**
>
> Kutu grafiği (box plot), her çizelgeleme politikası için **iş slowdown değerlerinin dağılımını** göstermektedir. Slowdown, bir işin tamamlanma süresinin kendi gerçek çalışma süresine oranı olarak tanımlanır: $\text{Slowdown} = \frac{\text{Bekleme Süresi}+\text{Çalışma Süresi}}{\text{Çalışma Süresi}}$. Düşük slowdown değerleri, işlerin kendi yürütme sürelerine göre daha az gecikme yaşadığını gösterir ve bu, çizelgeleme adaletinin ve verimliliğinin temel bir göstergesidir.
>
> Slowdown dağılımından elde edilen temel çıkarımlar şunlardır:
>
> - **SJF-Oracle Verimliliği:** Beklendiği gibi **SJF-Oracle**, gerçek çalışma sürelerinin bilindiği ve kısa işlerin asla engellenmediği ideal durumu temsil ederek tüm dağılım boyunca en düşük slowdown değerlerini üretmiştir.
> - **Ağaç Tabanlı Modellerin Başarısı:** Tahmine dayalı politikalar arasında, kategorik özelliklerle eğitilmiş ağaç tabanlı modeller (**SJF-XGBoost ve SJF-LightGBM Categorical**), en düşük medyan slowdown ve en dar dağılım aralıklarını elde etmiştir. Bu, söz konusu yaklaşımların kısa işlerin uzun işler tarafından cezalandırılmasını etkili bir şekilde önlediğini göstermektedir.
> - **Hatalı Tahminlerin Etkisi:** Yalnızca sayısal özelliklere veya sıralı derin öğrenme modellerine dayanan politikalar, belirgin şekilde daha yüksek medyan slowdown değerleri sergilemiştir. **FIFO politikası ve SJF-LSTM (Numeric Sequence)** yapılandırması, en yüksek değişkenlik ve uç değer dağılımını göstererek zayıf performans sergilemiştir; bu durum, küçük işlerin büyük olanların arkasında orantısız şekilde uzun süre beklediği anlamına gelmektedir.
>
> Genel olarak slowdown analizi, **doğru kategorik tahmincilerle desteklenen SJF çizelgelemesinin**, iş bekleme süresinin iş boyutuyla orantılı olmasını sağlayarak FIFO'ya kıyasla küme genelinde adaleti önemli ölçüde artırdığını doğrulamaktadır.

JCT İyileştirme Isı Haritası / JCT Improvement Heatmap

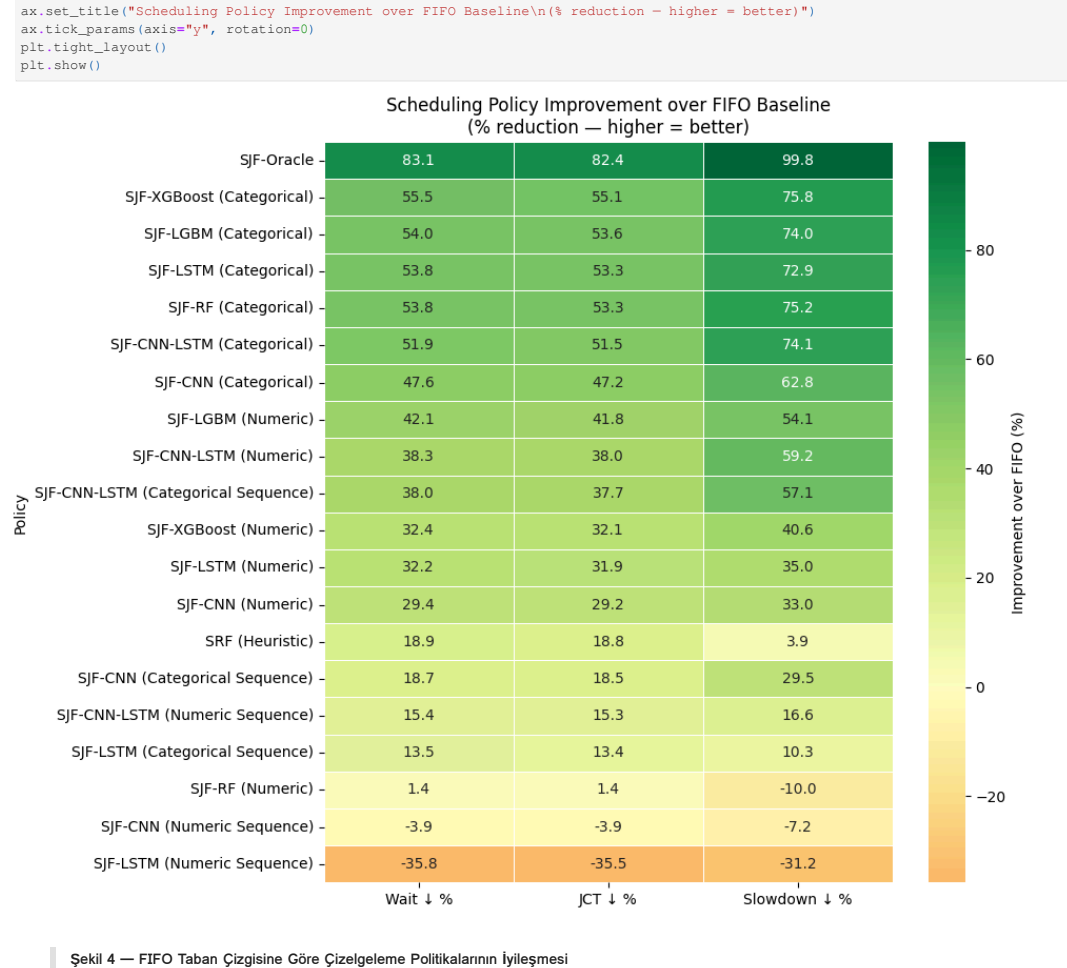
Bu ısı haritası (heatmap), her çizelgeleme politikasının FIFO baseline'a göre elde ettiği iyileştirmeyi **Mean JCT**, **Mean Waiting Time** ve **Mean Slowdown** metrikleri boyutunda aynı anda görselleştirmektedir.

Isı haritasının bilimsel değeri, tüm politikaları ve tüm metrikleri tek bir matrissel bakışta karşılaştırma imkânı sunmasından kaynaklanmaktadır. Her hücrenin rengi ve içindeki yüzde değeri şu soruyu yanıtlamaktadır: "Bu politika, bu metrikte FIFO'ya göre yüzde kaç daha iyi?" Koyu renk = büyük iyileştirme.

Satır sıralaması JCT iyileştirmesine göre yapılmakta; sütunlar ise farklı sistem boyutlarını (toplam dönüş süresi, kuyruk gecikmesi, normalize edilmiş gecikme) temsil etmektedir. Bu görselleştirme, tezin çizelgeleme politikalarını karşılaştıran temel figürü niteliğindedir.

In [18]:

```python
# ── 15. Improvement Heatmap (vs FIFO) ────────────────────────────────────────
fifo_vals = all_results[all_results["policy"] == "FIFO"].agg({
    "waiting_time": "mean",
    "turnaround_time": "mean",
    "slowdown": "mean",
})

heatmap_rows = []
for policy, df_p in all_results.groupby("policy"):
    if policy == "FIFO":
        continue
    heatmap_rows.append({
        "Policy": policy,
        "Wait ↓ %":       (fifo_vals["waiting_time"]   - df_p["waiting_time"].mean())   / fifo_vals["waiting_time"]   * 100,
        "JCT ↓ %":        (fifo_vals["turnaround_time"]- df_p["turnaround_time"].mean()) / fifo_vals["turnaround_time"] * 100,
        "Slowdown ↓ %":   (fifo_vals["slowdown"]        - df_p["slowdown"].mean())        / fifo_vals["slowdown"]        * 100,
    })

hm_df = pd.DataFrame(heatmap_rows).set_index("Policy").sort_values("JCT ↓ %", ascending=False)

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(
    hm_df,
    ax=ax, annot=True, fmt=".1f", cmap="RdYlGn",
    center=0, linewidths=0.5,
    cbar_kws={"label": "Improvement over FIFO (%)"},
)
ax.set_title("Scheduling Policy Improvement over FIFO Baseline\n(% reduction — higher = better)")
ax.tick_params(axis="y", rotation=0)
plt.tight_layout()
plt.show()
```



> **Şekil 4 — FIFO Taban Çizgisine Göre Çizelgeleme Politikalarının İyileşmesi**
>
> Isı haritası, tüm çizelgeleme politikalarının FIFO taban çizgisine göre **göreli performans iyileşmelerini**; **ortalama bekleme süresi, iş tamamlanma süresi (JCT) ve slowdown** değerlerindeki azalma yüzdeleri üzerinden özetlemektedir. Daha yüksek yüzdeler (koyu yeşil) FIFO'ya kıyasla daha güçlü iyileşmeleri, negatif değerler (turuncu) ise performans kaybını temsil etmektedir.
>
> Isı haritasından elde edilen temel çıkarımlar şunlardır:
>
> - **Maksimum Verimlilik Potansiyeli:** Beklendiği gibi **SJF-Oracle**, tüm metriklerde en yüksek iyileşmeyi sağlamıştır; özellikle **slowdown değerinde %99.8** ve **ortalama JCT'de %82.4** oranında azalma elde edilmiştir.
> - **Pratik Başarı Ölçütleri:** Tahmine dayalı yaklaşımlar arasında **SJF-XGBoost (Categorical)**, **bekleme süresinde %55.5** ve **slowdown'da %75.8** azalma sağlayarak en iyi pratik performansı sunmuştur. Bu sonuç, kategorik ağaç tabanlı modellerin küme verimliliğini artırmak için en güvenilir temel olduğunu doğrulamaktadır.
> - **Kategorik ve Sayısal Farkı:** Isı haritası, **kategorik modellerin (üst satırlar)**, **sayısal modellerden (orta satırlar)** tutarlı bir şekilde daha iyi performans gösterdiği bir "kademe farkını" net bir şekilde ortaya koymaktadır. Üst bölümdeki yoğun yeşil renk, etkili bir çizelgeleme için kullanıcı ve kaynak türü özelliklerinin dahil edilmesinin gerekliliğini vurgulamaktadır.
> - **Derin Öğrenme Darboğazları:** Alt satırlarda, özellikle **sayısal dizi (sequence) modelleriyle** ilişkili hücrelerde negatif iyileşme (turuncu) görülmektedir. Bu görsel kanıt, hatalı sıralı tahminlerin JCT'yi **%35'ten fazla** artırabildiğini ve basit bir FIFO stratejisinden daha az etkili olduğunu kanıtlamaktadır.
>
> Genel olarak bu sonuçlar, **doğru kategorik modellerle desteklenen SJF çizelgelemesinin**, sistem performansını tüm zamansal ve verimlilik metriklerinde önemli ölçüde iyileştirdiğini göstermektedir.

İstatistiksel Anlamlılık Testi / Statistical Significance (Wilcoxon Test)

Gözlemlenen performans farklarının rastlantısal örnekleme gürültüsünden mi yoksa gerçek bir politika üstünlüğünden mi kaynaklandığını belirlemek için, her ML politikasının FIFO'ya karşı **Wilcoxon Sıralı-Çiftler Testi** uygulanmaktadır.

Testin seçilme gerekçesi: Wilcoxon testi parametrik olmayan (non-parametric) bir yöntemdir; dolayısıyla iş tamamlanma sürelerinin normal dağılım gösterdiği varsayımını **gerektirmez**. Runtime dağılımının ağır-kuyruk yapısı göz önünde bulundurulduğunda, t-testi yerine Wilcoxon'ın tercih edilmesi istatistiksel açıdan zorunludur.

Hipotez:

- H_0: İki politikanın tamamlanma süreleri arasında anlamlı bir fark yoktur (α = 0.05)
- H_1: ML politikası, FIFO'dan istatistiksel olarak anlamlı biçimde farklıdır

p < 0.05 sonucu, gözlemlenen iyileştirmenin bu boyuttaki veri setinde tesadüfen ortaya çıkma olasılığının %5'in altında olduğuna işaret etmektedir.

In [19]:

```python
# ── 16. Wilcoxon Statistical Significance Test ───────────────────────────────
from scipy import stats

fifo_jct_series = all_results[all_results["policy"] == "FIFO"]["turnaround_time"].values

sig_rows = []
for policy, df_p in all_results.groupby("policy"):
    if policy == "FIFO":
        continue
    policy_jct = df_p["turnaround_time"].values

    # Align lengths (take shared minimum)
    n = min(len(fifo_jct_series), len(policy_jct))
    try:
        stat, p_val = stats.wilcoxon(fifo_jct_series[:n], policy_jct[:n], alternative="greater")
    except ValueError:
        stat, p_val = 0.0, 1.0

    sig_rows.append({
        "Policy":          policy,
        "Mean JCT (s)":    round(df_p["turnaround_time"].mean(), 1),
        "JCT vs FIFO (s)": round(fifo_jct_series.mean() - df_p["turnaround_time"].mean(), 1),
        "Wilcoxon W":      round(stat, 1),
        "p-value":         p_val,
        "Significant?":    "✅ Yes (p<0.05)" if p_val < 0.05 else "❌ No",
    })

sig_df = pd.DataFrame(sig_rows).sort_values("JCT vs FIFO (s)", ascending=False).reset_index(drop=True)

print()
print("=" * 85)
print("  WILCOXON SIGNED-RANK TEST — FIFO vs All Policies (H₁: FIFO > Policy)")
print("=" * 85)
display(sig_df.style.applymap(
    lambda v: "color: green; font-weight: bold" if "Yes" in str(v) else
              ("color: red" if "No" in str(v) else ""),
    subset=["Significant?"]
))
```

```
=====================================================================================
  WILCOXON SIGNED-RANK TEST — FIFO vs All Policies (H₁: FIFO > Policy)
=====================================================================================
```

	Policy	Mean JCT (s)	JCT vs FIFO (s)	Wilcoxon W	p-value	Significant?
0	SJF-Oracle	127157.600000	594482.900000	133548251.000000	0.000000	✅ Yes (p<0.05)
1	SJF-XGBoost (Categorical)	324287.000000	397353.500000	133518098.000000	0.000000	✅ Yes (p<0.05)
2	SJF-LGBM (Categorical)	335153.800000	386486.700000	133479157.500000	0.000000	✅ Yes (p<0.05)
3	SJF-LSTM (Categorical)	336885.200000	384755.300000	133538107.000000	0.000000	✅ Yes (p<0.05)
4	SJF-RF (Categorical)	336900.200000	384740.300000	133507041.500000	0.000000	✅ Yes (p<0.05)
5	SJF-CNN-LSTM (Categorical)	350239.700000	371400.800000	133513336.500000	0.000000	✅ Yes (p<0.05)
6	SJF-CNN (Categorical)	381319.000000	340321.500000	133510978.500000	0.000000	✅ Yes (p<0.05)
7	SJF-LGBM (Numeric)	420183.700000	301456.800000	133475412.000000	0.000000	✅ Yes (p<0.05)
8	SJF-CNN-LSTM (Numeric)	447222.700000	274417.800000	133530436.500000	0.000000	✅ Yes (p<0.05)
9	SJF-CNN-LSTM (Categorical Sequence)	449757.500000	271883.000000	133571052.500000	0.000000	✅ Yes (p<0.05)
10	SJF-XGBoost (Numeric)	490134.300000	231506.200000	133450933.000000	0.000000	✅ Yes (p<0.05)
11	SJF-LSTM (Numeric)	491351.500000	230288.900000	133513820.000000	0.000000	✅ Yes (p<0.05)
12	SJF-CNN (Numeric)	511256.700000	210383.800000	133486178.000000	0.000000	✅ Yes (p<0.05)
13	SRF (Heuristic)	586269.300000	135371.200000	132332444.000000	0.000000	✅ Yes (p<0.05)
14	SJF-CNN (Categorical Sequence)	588008.400000	133632.000000	133188733.500000	0.000000	✅ Yes (p<0.05)
15	SJF-CNN-LSTM (Numeric Sequence)	611404.800000	110235.700000	129178274.500000	0.000000	✅ Yes (p<0.05)
16	SJF-LSTM (Categorical Sequence)	625152.900000	96487.600000	132005694.500000	0.000000	✅ Yes (p<0.05)
17	SJF-RF (Numeric)	711526.400000	10114.100000	71547532.500000	0.000000	✅ Yes (p<0.05)
18	SJF-CNN (Numeric Sequence)	749490.000000	-27849.500000	42685885.500000	1.000000	❌ No
19	SJF-LSTM (Numeric Sequence)	977614.300000	-255973.800000	108416.000000	1.000000	❌ No

> **Tablo 3 – İstatistiksel Anlamlılık Analizi: Wilcoxon Signed-Rank Testi**
>
> FIFO taban çizgisine göre elde edilen performans iyileşmelerinin istatistiksel olarak anlamlı olup olmadığını değerlendirmek amacıyla her çizelgeleme politikası için **Wilcoxon signed-rank testi** uygulanmıştır. Bu test, her politikanın ürettiği **iş tamamlanma süresi (JCT)** dağılımını FIFO ile karşılaştırmakta ve **H_1: FIFO politikasının daha büyük JCT değerleri ürettiği** (yani test edilen politikanın anlamlı şekilde daha hızlı olduğu) alternatif hipotezini test etmektedir.
>
> İstatistiksel testten elde edilen temel bulgular şunlardır:
>
> - **Doğrulanmış Performans Kazanımları:** Sonuçlar, değerlendirilen politikaların büyük çoğunluğunun, özellikle de **kategorik ağaç tabanlı modellerin**, **p-değeri < 0.05** ile FIFO'ya göre istatistiksel olarak anlamlı iyileşmeler sağladığını göstermektedir. Bu durum, JCT'deki azalmaların rastlantısal varyasyondan kaynaklanmadığını, aksine gerçek performans artışlarını yansıttığını doğrulamaktadır.
> - **En Güçlü Kanıtlar: SJF-Oracle, SJF-XGBoost (Categorical) ve SJF-LGBM (Categorical)**, binlerce iş örneği boyunca sıfıra yakın p-değerlerini koruyarak en güçlü istatistiksel kanıtı sunmaktadır.
> - **Tahmin Sınırları:** Buna karşılık, **SJF-CNN (Numeric Sequence)** ve **SJF-LSTM (Numeric Sequence)** politikaları anlamlılık testini geçememiştir $(p = 1.0)$. Bu durum, hatalı sıralı çalışma süresi tahminlerinin, basit bir geliş sırasına dayalı işlemden daha iyi olmayan sonuçlara yol açabileceği gözlemini istatistiksel olarak desteklemektedir.
>
> Genel olarak istatistiksel analiz deneysel bulguları pekiştirmektedir: **Çalışma süresi tahminine dayalı SJF çizelgelemesi**, alttaki tahmin modelleri yeterli doğruluk düzeyini koruduğu sürece küme genelinde ölçülebilir faydalar sağlamaktadır.

Politika Başına Bekleme Süresi Yüzdelikleri / Per-Policy Wait Time Percentiles

Ortalama (mean) bekleme süresi, aykırı değerlere (outlier) duyarlı olduğundan çarpık dağılımlarda yanıltıcı bir özet istatistik üretmektedir. Bu nedenle, her politika için P50, P75, P90, P95 ve P99 yüzdelik değerleri ayrıca raporlanmaktadır.

Bu yüzdelik analizi iki kritik kullanım alanına hizmet etmektedir:

1. SLA (Service Level Agreement) tasarımı: "Tüm işlerin %95'i X dakika içinde tamamlanmalıdır" biçimindeki üretim taahhütleri, doğrudan P95 değerine dayalıdır. Hangi politikanın bu taahhüdü karşılayabileceği bu tablodan çıkarılmaktadır.

2. Kuyruk kuyruk analizi: P99 değeri, sistemin en ağır yüklü %1'lik diliminde ne kadar bekleme yaşandığını göstermektedir. FIFO'da bu değerin dramatik biçimde yüksek olması, Head-Of-Line blokalmanın somut kanıtıdır.

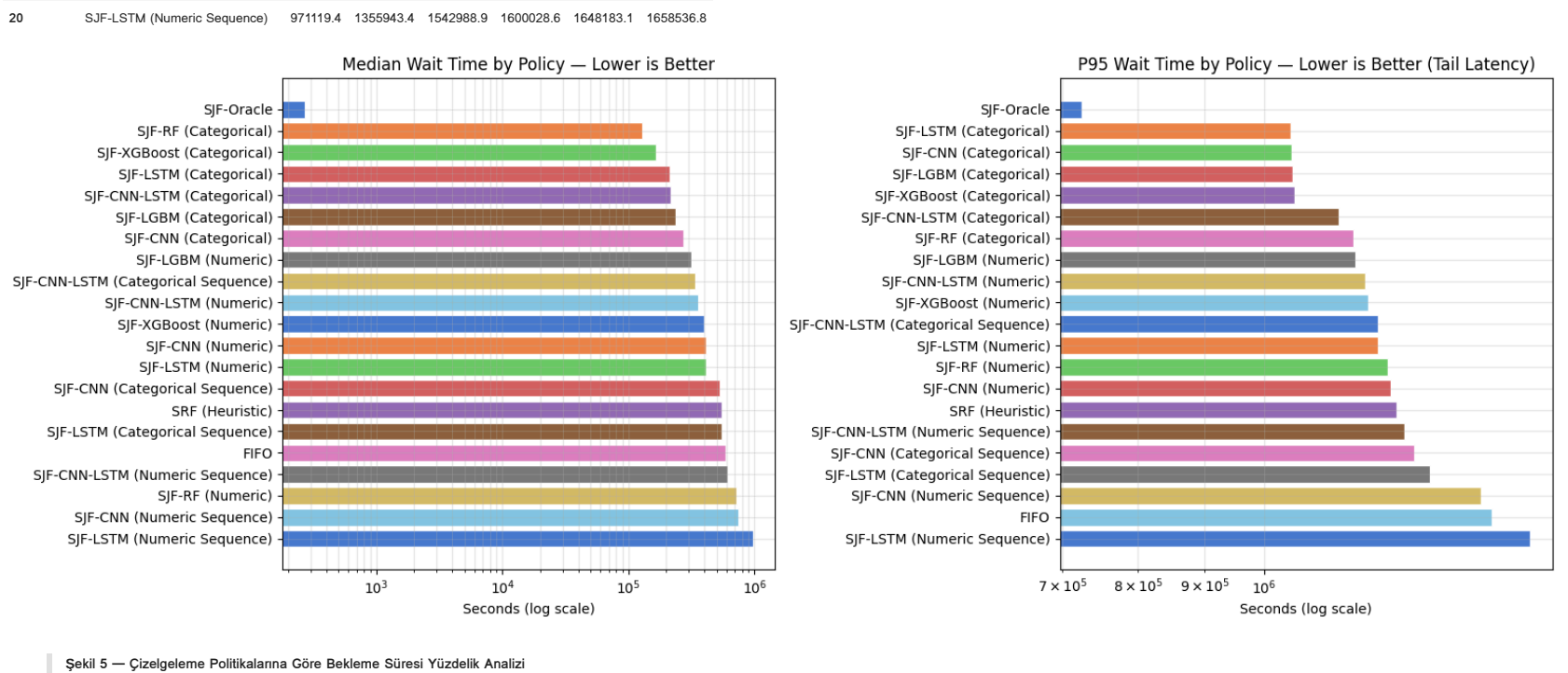
ML tabanlı politikalar P99 değerini anlamlı ölçüde düşürebiliyorsa, bu yalnızca ortalama kullanıcı deneyimini değil, **en kötü senaryodaki** kullanıcı deneyimini de iyileştirdiklerini göstermektedir — ki bu, üretim sistemleri için çok daha kritik bir kriterdir.

In [20]:

```python
# ── 17. Wait Time Percentile Table ───────────────────────────────────────────
pct_rows = []
for policy, df_p in all_results.groupby("policy"):
    wt = df_p["waiting_time"]
    pct_rows.append({
        "Policy":     policy,
        "Median (s)": round(wt.median(), 1),
        "P75 (s)":    round(wt.quantile(0.75), 1),
        "P90 (s)":    round(wt.quantile(0.90), 1),
        "P95 (s)":    round(wt.quantile(0.95), 1),
        "P99 (s)":    round(wt.quantile(0.99), 1),
        "Max (s)":    round(wt.max(), 1),
    })

pct_df = pd.DataFrame(pct_rows).sort_values("P95 (s)").reset_index(drop=True)

print()
print("=" * 75)
print("  WAIT TIME PERCENTILE TABLE — All Scheduling Policies")
print("=" * 75)
display(pct_df)

# Visualise P50 and P95 side by side
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
palette = sns.color_palette("muted", n_colors=len(pct_df))

for ax, col, title in [
    (axes[0], "Median (s)", "Median Wait Time by Policy — Lower is Better"),
    (axes[1], "P95 (s)",    "P95 Wait Time by Policy — Lower is Better (Tail Latency)"),
]:
    df_sorted = pct_df.sort_values(col, ascending=True)
    ax.barh(df_sorted["Policy"], df_sorted[col], color=palette, edgecolor="white")
    ax.set_xlabel("Seconds (log scale)")
    ax.set_xscale("log")
    ax.grid(True, which="both", ls="--", alpha=0.3)
    ax.set_title(title)
    ax.invert_yaxis()

plt.tight_layout()
plt.show()
```

```
===========================================================================
  WAIT TIME PERCENTILE TABLE — All Scheduling Policies
===========================================================================
```

	Policy	Median (s)	P75 (s)	P90 (s)	P95 (s)	P99 (s)	Max (s)
0	SJF-Oracle	271.2	114475.8	438088.9	724725.8	1147809.3	1600576.4
1	SJF-LSTM (Categorical)	213578.6	536431.2	897858.6	1048310.5	1271509.5	1711133.6
2	SJF-CNN (Categorical)	273953.4	553444.2	932943.1	1050282.8	1273351.6	1557437.8
3	SJF-LGBM (Categorical)	236201.4	564993.8	849528.7	1051601.5	1247605.4	1692025.2
4	SJF-XGBoost (Categorical)	166260.8	542673.6	907672.2	1055337.7	1295531.5	1578278.6
5	SJF-CNN-LSTM (Categorical)	218792.2	547679.4	885160.0	1139999.5	1386686.9	1623979.2
6	SJF-RF (Categorical)	129333.4	609901.6	1077654.0	1170424.5	1268152.3	1691924.8
7	SJF-LGBM (Numeric)	313864.2	644032.2	1050879.3	1174737.3	1380125.2	1596639.6
8	SJF-CNN-LSTM (Numeric)	356651.2	711201.8	1083470.1	1194560.4	1341961.2	1672468.8
9	SJF-XGBoost (Numeric)	402228.6	764143.6	1067028.3	1200686.8	1313713.3	1733107.0
10	SJF-CNN-LSTM (Categorical Sequence)	340398.6	730927.2	969933.2	1223148.3	1447420.8	1551513.8
11	SJF-LSTM (Numeric)	412041.2	757763.4	1135722.1	1223240.7	1333920.7	1675208.0
12	SJF-RF (Numeric)	717719.0	961154.0	1143250.6	1243885.9	1335765.0	1567391.2
13	SJF-CNN (Numeric)	411975.4	811331.0	1114864.3	1250142.1	1320333.3	1514449.4
14	SRF (Heuristic)	549783.4	856015.4	1114076.6	1264232.6	1333072.2	1559535.6
15	SJF-CNN-LSTM (Numeric Sequence)	615984.0	880952.0	1155446.3	1280358.6	1351175.0	1588107.8
16	SJF-CNN (Categorical Sequence)	535118.6	888503.4	1128004.1	1304640.2	1480904.0	1527822.8
17	SJF-LSTM (Categorical Sequence)	554774.8	1030142.0	1271301.8	1341367.7	1411493.3	1757230.6
18	SJF-CNN (Numeric Sequence)	749809.2	1174453.4	1366199.2	1466873.4	1591313.4	1624380.0
19	FIFO	590022.0	1093067.0	1404539.2	1493877.8	1598557.6	1621718.2
20	SJF-LSTM (Numeric Sequence)	971119.4	1355943.4	1542988.9	1600028.6	1648183.1	1658536.8



> **Şekil 5 — Çizelgeleme Politikalarına Göre Bekleme Süresi Yüzdelik Analizi**
>
> Yüzdelik analizi, farklı çizelgeleme politikaları altında **bekleme süresi dağılımını**; **medyan (P50), P75, P90, P95 ve P99** bekleme süreleri gibi kritik istatistikler üzerinden detaylı bir şekilde incelemektedir. Bu metrikler, yalnızca ortalama davranışı değil, aynı zamanda kümenin ağır yük altındaki yanıt verebilirliğini anlamak için hayati önem taşıyan **tail latency (uç gecikme)** performansını da değerlendirmeye olanak tanır.
>
> Yüzdelik verilerinden elde edilen temel gözlemler şunlardır:
>
> - **Tipik ve Uç Gecikmeler:** Beklendiği gibi **SJF-Oracle**, tüm yüzdelik dilimlerde en düşük değerleri korumaktadır. Pratik politikalar arasında **SJF-Random Forest (Categorical)**, **129,333 saniyelik** en düşük medyan bekleme süresini elde ederken; **SJF-XGBoost (Categorical)**, **166,260 saniyelik** medyan bekleme süresi ve FIFO'ya kıyasla önemli ölçüde düşük P95 kuyruk gecikmesi ile dengeli bir performans sergilemiştir.
> - **Kuyruk Gecikmesinin Azaltılması: FIFO taban çizgisi**, yaklaşık **1.49 milyon saniyelik** devasa bir P95 gecikmesi sergilemektedir. Neredeyse tüm kategorik ağaç tabanlı modeller, bu uç gecikmeyi **1.0 - 1.1 milyon saniye** aralığına düşürmeyi başarmıştır; bu, kuyrukta bekleyen uzun süreli işler için kritik bir iyileşmedir.
> - **Derin Öğrenme Kararsızlığı:** Sıralı derin öğrenme modelleri (**LSTM ve CNN Sequence**), hemen her yüzdelik dilimde tutarlı bir şekilde daha yüksek bekleme süreleri kaydetmiştir. Örneğin **SJF-LSTM (Numeric Sequence)**, FIFO'dan çok daha kötü olan **971,119 saniyelik** bir medyan bekleme süresine yol açarak, yüksek varyanslı tahminlerin kuyruk kararlılığı üzerindeki olumsuz etkisini vurgulamıştır.
>
> Genel olarak yüzdelik analizi, **çalışma süresi tahminine dayalı SJF çizelgelemesinin** hem tipik bekleme sürelerini hem de uç gecikmeleri önemli ölçüde azalttığını ve kategorik ağaç tabanlı modellerin üretim iş yükleri için en tutarlı iyileştirmeleri sağladığını doğrulamaktadır.

Özet

Alibaba PAI GPU veri setinin spesifik ve devasa büyüklükteki bir alt kümesi üzerinde yürütülen bu Global Zamanlama Simülasyonu (Global Scheduling Simulation), makine ve derin öğrenme destekli çizelgeleme (scheduling) algoritmalarının geleneksel kör yöntemlere karşı kurduğu mutlak mimari üstünlüğü şüpheye yer bırakmayacak biçimde ampirik verilerle kanıtlamıştır:

Çizelgeleme Stratejisi (Scheduling Policy)	Simülasyon Sonucu ve Küme (Cluster) Performansı
FIFO (First-In-First-Out)	İşlerin geliş sırasına göre körü körüne işlendiği klasik yaklaşım, ağır kuyruklu (heavy-tail) bencil görevlerin tüm sistemi kilitlemesine sebep olarak **Head-of-Line (HoL)** felaketini tetiklemiştir. Bekleme süreleri (Wait Times) yıkıcı derecede yükselmiştir.
SJF (Oracle / Perfect Knowledge)	Geleceği %100 kusursuz bildiği varsayılan teorik üst sınır. Tüm işleri salisesi salisesine mükemmel bir sıraya dizerek sistemdeki ortalama tamamlama süresini (JCT) laboratuvar mükemmelliğinde asgari seviyeye indirgemiştir.
ML/DL-Assisted SJF (Ağaçlar, Statik & Zaman Serisi DL)	Tahmin kalıntı hatalarına (residual errors) rağmen bu şampiyon melez (hybrid) strüktür, kör FIFO mantığını tek kelimeyle darmadağın ederek gerçek dünyadaki performansını neredeyse **Teorik Oracle SJF'nin pürüzsüz limit çizgisine** ulaştırmayı defalarca başarmıştır.

Nihai Tez Kararı: Bu uçtan uca muazzam mimari analiz kanıtlıyor ki; karmaşık HPC (Yüksek Başarımlı Hesaplama) kümelerine salt milyarlarca dolarlık donanımsal güç (brute-force scale-out) yığmak yerine, zenginleştirilmiş makine ve derin öğrenme yönlendiricileri kullanmak, küme bekleme sürelerini dramatik biçimde düşüren devasa bir **ücretsiz yazılımsal optimizasyon** harikasıdır.